



Escuela de
**Ingeniería Industrial
y Aeroespacial**
Toledo

Universidad de Castilla-La Mancha

Universidad de Castilla-La Mancha



TRABAJO FIN DE GRADO
Nº 26-D-225586

GRADO EN INGENIERÍA AEROESPACIAL

**Diseño e implementación de una herramienta de
software para la evaluación docente de
lenguajes de programación**

Iker Jinnian Blanco Moya

JULIO DE 2026

TRABAJO FIN DE GRADO
Nº 26-D-225586

GRADO EN INGENIERÍA AEROESPACIAL

**Diseño e implementación de una herramienta de
software para la evaluación docente de
lenguajes de programación**

Iker Jinnian Blanco Moya

JULIO DE 2026

Título: Diseño e implementación de una herramienta de software para la evaluación docente de lenguajes de programación

Autor: Iker Jinnian Blanco Moya

Grado: Grado en Ingeniería Aeroespacial

Tutor: David Muñoz Valero
Juan Moreno García

E-mail: IkerJinnian.Blanco@alu.uclm.es

Sitio web: https://github.com/UCLM-eiia-to/eiia_doc_tfg_latex

Dirección: UCLM — Escuela de Ingeniería Industrial y Aeroespacial
Campus Universitario de la Real Fábrica de Armas
45071 Toledo – Spain

© 2026 Iker Jinnian Blanco Moya

Esta obra está bajo una [Licencia Creative Commons Atribución 4.0 Internacional](#).

Muchos de los nombres usados por las compañías para diferenciar sus productos y servicios son reclamados como marcas registradas. Allí donde estos nombres aparezcan en este documento, y cuando el autor haya sido informado de esas marcas registradas, los nombres estarán escritos en mayúsculas o como nombres propios.

Agradecimientos

Valoro, tal vez no siempre ni todo lo que debería, la enorme suerte que he tenido en la vida. Ese azar favorable se refleja, por ejemplo, en el hecho de haber podido crecer en libertad y con medios materiales suficientes.

He tenido al alcance muchas oportunidades que me han permitido una dedicación completa a mi capacitación técnica y personal. Concibo esta etapa académica como una parte más del viaje hasta que la humanidad consiga una visión, la de un pálido punto azulado en el cielo de un desierto rojizo.

En agradecimiento a mis padres por hacer del azar una realidad material muy favorable.

*A quienes con rigor y, en ocasiones desde el anonimato, cimentaron el presente tecnológico
que hoy damos por sentado, demostrando que las fronteras del conocimiento son
únicamente temporales.*

*Y a las generaciones venideras, para que los desafíos futuros y la improbabilidad estadística
del éxito nunca sirvan de excusa para la inacción. Esto deberá ser simplemente un
recordatorio de la importancia que tiene seguir empujando los límites y convertir, mediante
la técnica y el rigor metodológico, lo improbable en inevitable.*

*Dedicado a quienes me han permitido llegar hasta aquí, apoyándome en todo lo posible
incluso cuando no he hecho méritos para merecerlo.*

Resumen

La integración de la enseñanza de la programación en titulaciones universitarias de ingeniería se ha consolidado como una competencia técnica fundamental. Sin embargo, la evaluación de estas competencias sigue enfrentándose a un cuello de botella logístico debido principalmente a la corrección manual de grandes volúmenes de código. Esto ralentiza el proceso y aumenta la carga de trabajo del profesorado. Para responder a esta necesidad, este **Trabajo Fin de Grado (TFG)** plantea el diseño de un prototipo web para la evaluación automática de ejercicios de Python en la asignatura de «Informática». El objetivo principal es reducir la corrección manual y proporcionar al estudiante una calificación inmediata basada en criterios deterministas, reproducibles y auditables.

La solución propuesta moderniza el método tradicional del examen en papel o mediante archivos sueltos, centralizando el proceso en una aplicación web donde el alumnado se identifica mediante los datos personales mínimos necesarios, como el nombre, los apellidos y un identificador académico o correo institucional. No se plantea el uso de documentos oficiales de identidad como identificadores visibles ni como campos operativos ordinarios del sistema. Una vez autenticado, el usuario accede a una prueba temporizada compuesta inicialmente por varios tipos de preguntas: ejercicios de rellenar huecos en bloques de código Python, ejercicios de detección y corrección de fallos sobre código editable, ejercicios de trazas con respuesta breve en lenguaje natural y preguntas tipo test con opciones desplegadas. En los formatos basados en código, el profesorado define plantillas, soluciones de referencia, casos de prueba o salidas esperadas. En las preguntas de trazas y tipo test, el sistema registra la respuesta del alumno y aplica, según proceda, rúbricas simples, comparación con opciones correctas o revisión docente posterior. Al enviar la prueba, o al finalizar el tiempo establecido si no la ha enviado antes, la aplicación procesa la entrega, almacena tanto las respuestas correctas como las incorrectas y devuelve una calificación automática cuando el tipo de pregunta permite una evaluación determinista.

Respecto al funcionamiento operativo de la plataforma, la **User Experience (UX)** se ha diseñado pensando en un estudiante que todavía se encuentra en una fase inicial de aprendizaje de la programación. El flujo de navegación comienza con una interfaz de bienvenida donde el alumno introduce sus credenciales básicas para identificarse. Una vez validado, el sistema despliega el examen activo y muestra una **UX** sencilla, con el enunciado de cada pregunta y el sistema despliega el examen activo y muestra una UX sencilla, con el enunciado de cada pregunta y una interfaz adaptada al tipo de ejercicio: huecos editables, bloque de código modificable, campo de respuesta breve o selector de opciones. Se incorpora además un temporizador descendente visible para que el estudiante pueda organizar su tiempo durante la prueba.

A nivel arquitectónico, la aplicación web se apoya en una estructura *full-stack* sencilla, compuesta por un *frontend* en **HyperText Markup Language (HTML)**, **Cascading Style Sheets (CSS)** y JavaScript nativo; un *backend* desarrollado con **FastAPI** y una capa de persistencia relacional. Esta elección prioriza la claridad técnica y la facilidad de despliegue. El *backend* centraliza la lógica del sistema, gestiona la autenticación básica, recibe las respuestas del alumnado, coordina el proceso de evaluación y devuelve los resultados al cliente.

La base de datos se plantea de forma compatible con SQLite en desarrollo y PostgreSQL en despliegues más exigentes.

El mayor desafío técnico del proyecto es la ejecución segura del código enviado por el alumno. Permitir que una aplicación web ejecute instrucciones proporcionadas por un usuario constituye un riesgo relevante de seguridad. Para mitigarlo, el sistema implementa una estrategia de aislamiento, comúnmente denominada *sandboxing*. El código completado por el alumno no se ejecuta libremente sobre el sistema operativo anfitrión, sino dentro de un entorno restringido donde se limita el tiempo de ejecución, los recursos disponibles y el acceso a la red. De esta manera, se evita tanto que un código malicioso intente afectar a la infraestructura como que errores no intencionados, como un bucle infinito, degraden el servicio.

La corrección automática se ha planteado con un enfoque determinista y auditable siempre que la naturaleza de la pregunta lo permite. La nota no depende de valoraciones generadas por inteligencia artificial, sino de mecanismos verificables: baterías de casos de prueba, comparación de salidas, análisis estático básico, validación estructural mediante **Abstract Syntax Tree (AST)** o coincidencia con opciones correctas. En las preguntas de código, el *backend* ensambla o recibe la solución escrita por el alumno, aplica validaciones previas y ejecuta el resultado en un entorno restringido. En las preguntas tipo test, compara la opción seleccionada con la clave definida por el profesorado. En las respuestas de trazas en lenguaje natural, el sistema conserva la respuesta y permite una revisión docente o semiautomática no vinculante, evitando que un modelo generativo determine por sí solo la calificación oficial.

Para el desarrollo del *frontend*, se ha priorizado la facilidad de uso y la respuesta fluida. El cliente web se comunica con la **Application Programming Interface (API)** del servidor mediante peticiones asíncronas, lo que permite enviar la prueba y recibir el resultado sin necesidad de recargar completamente la página si las respuestas se intercambian en **JavaScript Object Notation (JSON)**. Además, el cliente incorpora un mecanismo de supervisión disuasoria basado en eventos del navegador, como el cambio de pestaña, la pérdida de foco o la minimización de la ventana. Estos eventos se registran para auditoría y no generan por sí solos una penalización automática. Como ampliación condicionada a la configuración institucional, al cumplimiento normativo y a la autorización técnica del navegador, el sistema podrá iniciar o activar la recogida de evidencias asociadas al evento, como grabación de pantalla, audio de micrófono y cámara frontal, con el único fin de que el profesorado pueda revisar posteriormente la incidencia. La decisión efectiva sobre el tiempo disponible, la validez del envío, la existencia de penalización y la nota final permanece siempre en el servidor y bajo revisión docente.

La configurabilidad es otro de los pilares fundamentales del diseño. El sistema se ha concebido como un prototipo reproducible y ampliable, capaz de gestionar un conjunto inicial de ejercicios sin necesidad de rehacer la aplicación para cada convocatoria. Esto permite al profesorado definir preguntas, plantillas de código y casos de prueba, manteniendo separadas la lógica de evaluación y la presentación al alumno. Esta organización asegura la vida útil del proyecto más allá de una única sesión docente y abre la puerta a futuras ampliaciones.

En cuanto a la integridad de la prueba, el sistema combina medidas de supervisión ligera en el cliente con control efectivo en el *backend*. El navegador puede detectar ciertos eventos relevantes como cambios de pestaña y registrarlos con fines de auditoría. Sin embargo, el diseño del proyecto asume que la seguridad real no debe descansar únicamente

sobre el lado del cliente. Por ello, el servidor valida el intento, ejecuta la corrección, registra el resultado y conserva de forma persistente la información asociada a cada entrega.

La plataforma pretende mejorar la dinámica de enseñanza y evaluación al reducir la corrección repetitiva y facilitar retroalimentación inmediata o diferida según el tipo de pregunta. Al eliminar parte del intervalo entre la realización de la prueba y la obtención de resultados, el alumnado recibe información más rápida sobre sus aciertos y errores. Además, se contempla el uso auxiliar de modelos de lenguaje para generar, a partir de datos anonimizados o seudonimizados, pequeños informes de retroalimentación: uno individual para el estudiante, centrado en su examen particular, y otro agregado para el profesorado, orientado a identificar patrones de rendimiento, preguntas problemáticas y errores frecuentes. Estos informes no sustituyen al motor de corrección ni modifican la calificación oficial. La objetividad también mejora, ya que todos los estudiantes son evaluados mediante el mismo conjunto de pruebas y bajo el mismo criterio de puntuación.

Técnicamente, el proyecto integra de forma coherente varias áreas de la ingeniería del software aplicadas al ámbito educativo. Incluye desarrollo de interfaces, diseño de **Application Programming Interface (API)**, persistencia relacional, ejecución segura de código y validación automática. La elección de tecnologías relativamente sencillas y conocidas facilita tanto el mantenimiento como la comprensión global del sistema. Además, el trabajo incorpora pruebas automáticas, scripts de inicialización, archivos en formato **Comma-Separated Values (CSV)** y una canalización de **Continuous Integration (CI)**, lo que permite demostrar el funcionamiento del prototipo de manera reproducible.

Como resultado, se obtiene una base técnica sólida para la evaluación automática de ejercicios de Python en un contexto universitario. Más que un producto cerrado y definitivo, la aplicación es un prototipo funcional, reproducible y razonablemente seguro, capaz de cubrir el ciclo esencial de identificación, realización del examen, corrección, almacenamiento de resultados y consulta posterior por parte del profesorado mediante un panel básico de resultados. En este sentido, el trabajo no solo propone una mejora concreta para la asignatura de Informática, sino que establece un punto de partida realista para futuras ampliaciones, como el endurecimiento adicional del *sandbox* o la extensión a otros formatos de ejercicios y otros lenguajes de programación.

Abstract

The integration of programming instruction into university engineering degree programmes has become established as a fundamental technical competence. However, the assessment of these competences continues to face a logistical bottleneck, due mainly to the manual marking of large volumes of code. This slows the process and increases the workload of the teaching staff. In response to this need, this [Trabajo Fin de Grado \(TFG\)](#) presents the design of a web based prototype for the automated assessment of Python exercises in the course “Informatics”. Its main objective is to reduce manual marking and to provide students with an immediate grade based on deterministic, reproducible, and auditable criteria.

The proposed solution modernises the traditional method of paper based examinations or separate file submissions by centralising the process in a web application where students identify themselves by means of only the minimum personal data required, such as their first name, surnames, and an academic identifier or institutional email address. The use of official identity documents is not envisaged either as visible identifiers or as ordinary operational fields within the system. Once authenticated, the user accesses a timed examination initially composed of several types of question: exercises requiring the completion of missing fragments in Python code blocks, exercises based on the detection and correction of faults in editable code, code tracing exercises requiring a short answer in natural language, and multiple choice questions with drop down options. In code based formats, the teaching staff define templates, reference solutions, test cases, or expected outputs. In code tracing and multiple choice questions, the system records the student’s response and applies, where appropriate, simple rubrics, comparison against the correct options, or subsequent consultation review by the teaching staff. When the examination is submitted, or when the allotted time expires if it has not been submitted beforehand, the application processes the submission, stores both correct and incorrect responses, and returns an automatic grade whenever the nature of the question allows deterministic assessment.

With regard to the operational behaviour of the platform, the [User Experience \(UX\)](#) has been designed for students who are still at an early stage of learning programming. The navigation flow begins with a welcome interface in which the student enters their basic credentials for identification. Once validated, the system displays the active examination and presents a simple [UX](#), showing the wording of each question together with the corresponding code block and the editable fields to be completed. A visible countdown timer is also included so that the student can manage their time during the examination.

At the architectural level, the web application is based on a simple *full stack* structure composed of a *frontend* implemented in [HyperText Markup Language \(HTML\)](#), [Cascading Style Sheets \(CSS\)](#), and native JavaScript; a *backend* developed with *FastAPI*; and a relational persistence layer. This choice prioritises technical clarity and ease of deployment. The *backend* centralises the system logic, manages basic authentication, receives student responses, coordinates the assessment process, and returns the results to the client. The database is designed to remain compatible with SQLite during development and PostgreSQL in more demanding deployments.

The greatest technical challenge of the project is the secure execution of the code submitted by the student. Allowing a web application to execute instructions provided by a user constitutes a significant security risk. To mitigate this, the system implements an isolation strategy commonly known as *sandboxing*. The code completed by the student is not executed freely on the host operating system, but within a restricted environment in which execution time, available resources, and network access are limited. In this way, the system prevents both malicious code from attempting to affect the infrastructure and unintentional errors, such as an infinite loop, from degrading the service.

Automatic marking has been designed according to a deterministic and auditable approach whenever the nature of the question allows it. The grade does not depend on assessments generated by artificial intelligence, but on verifiable mechanisms: sets of test cases, output comparison, basic static analysis, structural validation through **Abstract Syntax Tree (AST)**, or matching against the correct options. In code questions, the *backend* assembles or receives the solution written by the student, applies prior validations, and executes the result in a restricted environment. In multiple choice questions, it compares the selected option with the answer key defined by the teaching staff. For code tracing responses written in natural language, the system stores the response and allows review by the teaching staff or non binding semi automatic review, thereby preventing a generative model from determining the official grade on its own.

For the development of the *frontend*, priority has been given to ease of use and fluid responsiveness. The web client communicates with the server side **Application Programming Interface (API)** through asynchronous requests, which makes it possible to submit the examination and receive the result without fully reloading the page if responses are exchanged in **JavaScript Object Notation (JSON)**. In addition, the client incorporates a deterrent monitoring mechanism based on browser events such as tab switching, loss of focus, or window minimisation. These events are recorded for auditing purposes and do not, by themselves, result in an automatic penalty. As an extension subject to institutional configuration, regulatory compliance, and the technical authorisation granted by the browser, the system may initiate or activate the collection of evidence associated with the event, such as screen recording, microphone audio, and front camera footage, solely so that the teaching staff may subsequently review the incident. The effective decision regarding the time available, the validity of the submission, the existence of any penalty, and the final grade always remains on the server and under review by the teaching staff.

Configurability is another fundamental pillar of the design. The system has been conceived as a reproducible and extensible prototype capable of managing an initial set of exercises without requiring the application to be rebuilt for each examination session. This allows the teaching staff to define questions, code templates, and test cases while keeping the evaluation logic separate from the presentation shown to the student. This organisation ensures the useful life of the project beyond a single teaching session and opens the door to future extensions.

With regard to examination integrity, the system combines lightweight supervision measures on the client side with effective control in the *backend*. The browser can detect certain relevant events, such as tab switching, and record them for auditing purposes. However, the design of the project assumes that real security must not rely solely on the client side. For this reason, the server validates the attempt, executes the marking process, records the result, and persistently stores the information associated with each submission.

The platform is intended to improve the dynamics of teaching and assessment by re-

ducing repetitive marking and facilitating immediate or deferred feedback depending on the type of question. By removing part of the interval between completion of the examination and receipt of the results, students receive faster information about their correct and incorrect responses. In addition, the auxiliary use of language models is envisaged in order to generate, from anonymised or pseudonymised data, short feedback reports: one individual report for the student, focused on that student's own examination, and one aggregated report for the teaching staff, aimed at identifying patterns of performance, problematic questions, and frequent errors. These reports do not replace the marking engine and do not modify the official grade. Objectivity is also improved, since all students are assessed using the same set of tests and the same scoring criterion.

From a technical perspective, the project coherently integrates several areas of software engineering applied to the educational domain. It includes interface development, **Application Programming Interface (API)** design, relational persistence, secure code execution, and automatic validation. The choice of relatively simple and well known technologies facilitates both maintenance and overall understanding of the system. In addition, the work incorporates automated tests, initialisation scripts, files in **Comma-Separated Values (CSV)** format, and a **Continuous Integration (CI)** pipeline, which makes it possible to demonstrate the operation of the prototype in a reproducible manner.

As a result, the work provides a solid technical basis for the automated assessment of Python exercises in a university context. Rather than being a closed and definitive product, the application is a functional, reproducible, and reasonably secure prototype capable of covering the essential cycle of identification, examination completion, marking, result storage, and subsequent consultation by the teaching staff through a basic results dashboard. In this sense, the project not only proposes a concrete improvement for the course in Informatics, but also establishes a realistic starting point for future extensions, such as further hardening of the *sandbox* or extension to other exercise formats and other programming languages.

Índice general

1	Introducción	1
1.1	Organización de la memoria	1
1.2	Repositorio de información	2
2	Motivación y antecedentes	3
2.1	Introducción	3
2.2	Corrección de exámenes a gran escala	3
2.2.1	Contexto global	3
2.2.2	Deficiencias de la corrección manual	3
2.2.3	Requisitos específicos de la evaluación sumativa	4
2.3	La importancia del lenguaje de programación Python	4
2.3.1	Relevancia actual de Python	4
2.3.2	Retos técnicos específicos de Python para la evaluación automática	5
2.4	Metodología de la revisión narrativa estructurada	6
2.5	Evolución histórica de la evaluación automática de código	6
2.6	Sistemas académicos clásicos	7
2.6.1	WebCAT	7
2.6.2	Marmoset	7
2.6.3	BOSS	7
2.6.4	Submit	7
2.6.5	testBench	8
2.7	Integración con Sistemas de Gestión del Aprendizaje (LMS)	8
2.7.1	VPL (Virtual Programming Lab)	8
2.7.2	CodeRunner	8
2.8	Soluciones comerciales y jueces en línea	9
2.8.1	Gradescope	9
2.8.2	Jueces en línea	9
2.9	Retroalimentación pedagógica avanzada y ejercicios de completar código	10
2.9.1	El problema del «string matching»	10
2.9.2	Modalidades de pregunta en evaluación introductoria de Python	10
2.9.3	Evaluación semántica mediante Árboles de Sintaxis Abstracta (AST)	10
2.10	El rol y las limitaciones de los modelos de lenguaje (LLM)	11

2.11 Seguridad durante la ejecución e integridad académica	11
2.11.1 Sandboxing y protección del servidor	11
2.11.2 Supervisión del entorno del cliente	12
2.11.3 Cumplimiento con el RGPD	12
2.12 Análisis comparativo del estado del arte	13
2.13 Lagunas identificadas y justificación del proyecto	14
3 Objetivos	17
Objetivo 1. Arquitectura <i>full-stack</i>	17
Objetivo 2. Base de datos estructurada.	18
Objetivo 3. Motor de ejecución segura (<i>sandbox</i>).	19
Objetivo 4. Interfaz de examen.	20
Objetivo 5. Validación del sistema integrado.	21
4 Análisis de alternativas y decisiones de diseño	25
4.1 Criterios de decisión	25
4.2 Modelo de persistencia de datos	25
4.2.1 Alternativas de organización (una o varias bases de datos)	25
4.2.2 Alternativas de motor (SQLite, PostgreSQL, NoSQL)	25
4.2.3 Decisión tomada y su justificación	25
4.3 Motor de ejecución segura de código (<i>sandbox</i>)	26
4.3.1 Ejecución sin aislamiento	26
4.3.2 Aislamiento a nivel de aplicación (<i>subprocess + rlimit</i>)	26
4.3.3 Aislamiento a nivel de sistema operativo (Docker Engine o Docker Desktop)	26
4.3.4 Alternativas avanzadas (<i>gVisor, Firecracker, nsjail, WebAssembly</i>)	26
4.3.5 Decisión tomada y su justificación	26
4.4 Validación estática previa del código del alumno	26
4.4.1 Filtro previo a la ejecución	26
4.4.2 Alternativas	26
4.4.3 Decisión tomada	26
4.5 Modalidad de acceso y entorno de examen	26
4.5.1 Aplicación web o instalable	26
4.5.2 Lockdown browsers (<i>Safe Exam Browser, Respondus</i>)	26
4.5.3 Arranque restringido por hardware (<i>pendrive/Live USB</i>)	26
4.5.4 Modificación de BIOS/UEFI	26
4.5.5 Supervisión ligera en cliente y control efectivo en servidor	26
4.6 Resumen de decisiones y trazabilidad con los objetivos	26
5 Desarrollo del TFG	27
5.1 Metodología de trabajo	27

5.1.1	Enfoque metodológico	27
5.2	Planificación y Fases del Proyecto	27
5.2.1	Fase 1: Arquitectura y Modelo de Datos	27
5.2.2	Fase 2: Motor de Evaluación y Sandbox	27
5.2.3	Fase 3: Interfaz y Auditoría	27
5.2.4	Fase 4: Validación y Despliegue	27
5.3	Ejecución del Proyecto y Desviaciones	27
5.3.1	Problemas encontrados en el aislamiento de código	27
5.3.2	Desviaciones en la persistencia y concurrencia	27
5.3.3	Retos en el frontend sin frameworks	27
6	Resultados y discusión	29
6.1	Resultado funcional global	29
6.1.1	Descripción del prototipo final	29
6.1.2	Flujo de valor para el docente y el alumno	29
6.2	Validación del sandbox y la seguridad	29
6.2.1	Eficacia del análisis estático (AST)	29
6.2.2	Aislamiento a nivel de Sistema Operativo	29
6.3	Pruebas de rendimiento y concurrencia	29
6.3.1	Metodología de la prueba de carga	29
6.3.2	Resultados de latencia y throughput	29
6.3.3	Discusión sobre el cuello de botella de base de datos	29
6.4	Contribuciones originales y metodológicas	29
6.4.1	Motor de corrección «Batch» determinista	29
6.4.2	Privacidad por diseño (Privacy by Design)	29
6.4.3	Auditoría de integridad académica no invasiva	29
6.5	Resultados negativos y limitaciones técnicas	29
6.5.1	Insuficiencia del «Subprocess» local para entornos compartidos	29
6.5.2	Limitaciones del soporte inicial multiformato	29
6.5.3	Falsos negativos en la comparación de salidas	30
7	Conclusiones	31
7.1	Análisis del cumplimiento de los objetivos	31
7.1.1	Grado de cumplimiento del objetivo 1	31
7.1.2	Grado de cumplimiento del objetivo 2	31
7.1.3	Grado de cumplimiento del objetivo 3	31
7.1.4	Grado de cumplimiento del objetivo 4	31
7.1.5	Grado de cumplimiento del objetivo 5	31
7.2	Trabajo no realizado y limitaciones del alcance	31
7.2.1	Integración con LMS (Moodle/Canvas)	31
7.2.2	Estudio de usabilidad formal (SUS)	31

7.2.3	Soporte para múltiples lenguajes	31
7.3	Líneas de trabajo futuro	31
7.3.1	Mejora de la retroalimentación pedagógica mediante IA no evaluativa	31
7.3.2	Ampliación avanzada del soporte multiformato de preguntas	31
7.3.3	Despliegue en la nube y escalado horizontal	32
7.3.4	Analítica del proceso de escritura de código	32
7.4	Valoración personal y cierre	32
A		33
A.1	Declaración responsable sobre autoría y uso ético de Inteligencia Artificial Generativa	33
A.2	Texto	34
A.2.1	Listas numeradas y con viñetas	35
A.2.2	Acrónimos	35
A.3	Figuras	35
A.4	Tablas	39
A.5	Ecuaciones	41
A.6	Bibliografía, citas y referencias	42
A.6.1	Lo que toda referencia debe tener	43
A.7	Hojas de datos	43
A.8	Código fuente	46
A.9	Compilación local	48
A.9.1	Configurar el entorno	48
A.9.2	Configurar TeXStudio	49
Bibliografía		51

Índice de cuadros

2.1	Análisis comparativo de sistemas de evaluación automática de código. . . .	14
A.1	Resultados de la simulación	39
A.2	Tabla muy sencilla.	40
A.3	Fusionando celdas.	40
A.4	Un ejemplo de tabla larga	40

Índice de figuras

A.1	Figurita ejemplo	36
A.2	Figurita ejemplo 2	36
A.3	Figurita ejemplo 3. Extraída de la plantilla de TFG de Fernando Castillo. ©2018 Fernando Castillo. Reproducida con permiso.	37
A.4	Figura ejemplo que ocupa todo el ancho del texto.	38
A.5	Figura ejemplo de rotación.	38
A.6	Matriz de figuras	39
a	Figurita 1	39
b	Figurita 2	39
c	Figurita 3	39
A.7	Figura ejemplo. Tomada de hoja de catálogo de motores DC con escobillas de grafito de Maxon © 2014 Maxon Motors. Reproducida con permiso. . . .	45
A.8	Configuración de la compilación en TeXStudio	49
A.9	Configuración de la orden <i>Latexmk</i> en TeXStudio	49

Lista de símbolos

Mayúsculas

- T — Temperatura (°C).
 V — Volumen (m³).

Minúsculas

- c — Velocidad de la luz en el vacío (m/s). La velocidad de la radiación electromagnética es independiente de la velocidad del emisor.
 i — Raíz de menos uno (-).

Letras griegas

- α — El principio de todo (-).
 π — Pastel en inglés (-).

Lista de acrónimos

ACM

Association for Computing Machinery. [6](#)

API

Application Programming Interface. [i](#), [viii](#), [ix](#), [xii](#)

AST

Abstract Syntax Tree. [viii](#), [xii](#), [5](#), [10](#), [14](#)

CI

Continuous Integration. [i](#), [ix](#), [22](#)

CPU

Central Processing Unit. [8](#), [11](#), [19](#)

CRA

Computing Research Association. [3](#)

CSS

Cascading Style Sheets. [vii](#), [xi](#), [5](#), [18](#)

CSV

Comma-Separated Values. [i](#), [ix](#)

DNI

Documento Nacional de Identidad. [13](#)

DoD

Definition of Done. [17](#)

DTO

Data Transfer Object. [21](#)

EIIA-To

Escuela de Ingeniería Industrial y Aeroespacial de Toledo. [33](#)

ESO

Educación Secundaria Obligatoria. [3](#)

HTML

HyperText Markup Language. [vii](#), [xi](#), [5](#), [18](#)

HTTP

HyperText Transfer Protocol. [17](#)

IA

Inteligencia Artificial. 11, 12, 33

IEEE

Institute of Electrical and Electronics Engineers. 6

JSON

JavaScript Object Notation. viii, xii

LLM

Large Language Model. 11, 13, 15, 19, 30, 31, 33

LMS

Learning Management System. 7, 8, 13, 14

LOPDGDD

Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales. 12

LTI

Learning Tools Interoperability. 9

MAC

Mandatory Access Control. 12

ORM

Object-Relational Mapping. 18

PII

Personally Identifiable Information. 12

PYPL

PopularitY of Programming Language. 4, 5

RGPD

Reglamento General de Protección de Datos. 9, 12, 21

SaaS

Software as a Service. 9

SQL

Structured Query Language. 5

SUS

System Usability Scale. 22

TFG

Trabajo Fin de Grado. vii, xi, 2–4, 6, 11, 14, 17, 33

UCLM

Universidad de Castilla-La Mancha. 33

UI

User Interface. 17, 21

UPC

Universitat Politècnica de Catalunya. 8

UVa

Universidad de Valladolid. 9

UX

User Experience. vii, xi, 14, 21

VPL

Virtual Programming Lab. 8

Capítulo 1

Introducción

La programación es una competencia esencial en la ingeniería actual, y, en particular, en el Grado en Ingeniería Aeroespacial. En la asignatura de Informática, los alumnos adquieren la lógica necesaria para resolver problemas mediante lenguajes como Python. Sin embargo, la evaluación eficiente e imparcial de esta capacidad plantea dificultades logísticas.

En la actualidad, el proceso de corrección de exámenes de programación supone una elevada carga de trabajo para el profesorado. Es una tarea repetitiva, lo que incrementa la probabilidad de cometer errores durante la evaluación debido a la fatiga del evaluador. Además, la realización de pruebas de programación en equipos informáticos sin un entorno controlado compromete la seguridad de la prueba. Aunque existen plataformas generalistas orientadas al aprendizaje de la programación, como Codecademy [Cod26] o Coursera [Cou26], estas herramientas no se adaptan directamente a las restricciones propias de una evaluación universitaria sumativa.

Existe, por tanto, una necesidad específica no cubierta plenamente en el entorno de la asignatura. Se necesita un sistema hecho a medida para realizar pruebas de progreso en un entorno controlado, con soporte inicial para varios tipos de preguntas. Las pruebas contendrán ejercicios de completar huecos en código Python, corregir fallos en bloques de código, realizar trazas mediante una explicación breve y responder preguntas tipo test. El sistema debe registrar eventos relevantes del navegador, reducir, sin eliminar completamente, el riesgo de acceso a fuentes externas y aumentar la eficiencia del proceso. Además, debe ser capaz de ejecutar código del alumno cuando la pregunta lo requiera, contrastarlo con casos de prueba o salidas esperadas, registrar todas las respuestas enviadas, tanto correctas como incorrectas, y generar una calificación automática cuando exista un criterio determinista. El desarrollo de esta herramienta de software contribuirá a reducir la carga de trabajo asociada a la corrección manual y proporcionará al estudiante información más rápida sobre su desempeño, abriendo la posibilidad a una práctica deliberada más efectiva.

1.1 Organización de la memoria

Este documento se organiza en los siguientes capítulos:

Capítulo 2

El incremento sostenido de estudiantes en disciplinas técnicas ha evidenciado la ineficiencia de la evaluación manual de código. Esto ha impulsado el desarrollo de plataformas de corrección automática que logran evaluar ejercicios en tiempo real garantizando una calificación objetiva y escalable.

Capítulo 3

El objetivo es desarrollar un prototipo funcional de aplicación web para automatizar la corrección. También se delimitan sus principales limitaciones, las cuales son la evaluación restringida a Python, un diseño funcional orientado al prototipo y un alcance ajustado al marco temporal de un TFG.

Capítulo 4

Se analizan las alternativas técnicas consideradas y se justifican las decisiones de diseño adoptadas para la persistencia de datos, la ejecución segura del código, la validación estática, la supervisión del entorno de examen y el soporte inicial de varios tipos de preguntas.

Capítulo 5

Se muestra el proceso completo de desarrollo del TFG, con la metodología de trabajo empleada y las diferentes etapas en las que se ha llevado a cabo.

Capítulo 6

Se muestran los resultados obtenidos y se analizan las pruebas realizadas en relación con los objetivos del proyecto.

Capítulo 7

Se muestran las conclusiones del proyecto y se abre la puerta a futuras líneas de trabajo.

Anexos

Se adjunta información técnica útil que facilita la reproducción de los resultados y la revisión técnica del trabajo, complementando la información del cuerpo de este documento aunque no sea estrictamente necesaria para entender el trabajo completo.

Bibliografía

Se incluyen todas las referencias bibliográficas utilizadas durante el desarrollo del proyecto.

1.2 Repositorio de información

Todo el material generado durante la ejecución de este proyecto está disponible en el repositorio https://github.com/UCLM-eiia-to/eiia_doc_tfg_latex.git. El material incluye el código $\text{\LaTeX}$ del presente documento, el código fuente de los programas realizados o modificados, y todos los datos generados en la evaluación de resultados.

Capítulo 2

Motivación y antecedentes

2.1 Introducción

Este capítulo se estructura partiendo del análisis del crecimiento en la enseñanza de la programación (sección 2.2) y la justificación técnica de Python (sección 2.3). A continuación, se detalla la metodología de revisión narrativa estructurada, inspirada en una revisión sistemática de literatura y adaptada al alcance de un TFG (sección 2.4), para explorar la evolución histórica y el estado del arte de los sistemas de evaluación automática (secciones 2.5 a 2.10). Finalmente, se contrastan estas tecnologías frente a los requisitos de integridad académica (sección 2.11), sintetizando las lagunas no resueltas que motivan el presente proyecto (secciones 2.12 y 2.13).

2.2 Corrección de exámenes a gran escala

2.2.1 Contexto global

La programación es una competencia fundamental en educación universitaria. Según el *Taulbee Survey 2022* de la [Computing Research Association \(CRA\)](#), el número de estudiantes matriculados en programas de informática en universidades norteamericanas se triplicó entre 2006 y 2021. Este crecimiento equivale aproximadamente a una tasa geométrica media anualizada de un 7.6 % durante 15 años, lo que la [Computing Research Association \(CRA\)](#) [Com22] califica como estructural.

En Europa, el programa digital *Digital Decade 2030* [Eur23] fija como objetivo que al menos el 80 % de las personas adultas disponga de competencias digitales básicas antes de 2030.

A nivel nacional, el Real Decreto 217/2022 [22] incorpora competencias vinculadas al *pensamiento computacional, la programación y la robótica* en el currículo de la [Educación Secundaria Obligatoria \(ESO\)](#). En las universidades españolas, esta tendencia se materializa en una o varias asignaturas de informática en los grados de ingeniería.

2.2.2 Deficiencias de la corrección manual

El crecimiento descrito ha provocado una carga estructural creciente en el proceso de evaluación. En el modelo tradicional con una corrección manual, un ejercicio de programación exige que el docente lea un volumen elevado de exámenes en papel. Si se utiliza el ordenador, el docente debe descargar el fichero de código de cada estudiante, configurar un

entorno local compatible, ejecutar el programa, analizar las salidas e identificar los fallos.

Queirós y Leal [QL12] describen la corrección manual de ejercicios de programación como un proceso costoso para el profesorado, especialmente cuando aumenta el número de estudiantes. A modo de estimación, si se requieren unos 10 minutos por estudiante para un ejercicio de dificultad media, en clases de 70 o más alumnos esto compromete más de 11 horas de trabajo mecánico por cada prueba evaluable. Ese tiempo dedicado a la corrección no se puede dedicar a la preparación de material didáctico y tutorías.

La evaluación manual [QL12] es lenta y consume mucho tiempo. Además, cuando intervienen varios correctores, puede aparecer variabilidad interevaluador [MBKS25] incluso con rúbricas compartidas. Dichas restricciones se ven agravadas por la fatiga cognitiva derivada de sesiones extensas de corrección, que propicia fallos sistemáticos en la aplicación de criterios rubricados. También aumenta significativamente la variabilidad empíricamente constatada [IAKS10] en la puntuación de errores lógicos idénticos, tanto en el mismo evaluador como entre distintos evaluadores.

Además, el intervalo entre la realización del examen y la recepción de los resultados en dicha prueba alarga, cuando no interrumpe completamente, el bucle de retroalimentación que la literatura pedagógica considera esencial para la consolidación del aprendizaje técnico, la llamada *práctica deliberada*.

2.2.3 Requisitos específicos de la evaluación sumativa

La automatización de la tarea de evaluación surge como respuesta natural a esa situación. Sin embargo, es relevante diferenciar dos métodos de evaluación distintos, cada uno con sus propios requisitos.

En primer lugar, la evaluación formativa con prácticas, entregas y ejercicios continuos. Esta modalidad puede realizarse de forma asíncrona, con varios intentos y retroalimentación instantánea.

En segundo lugar, la evaluación sumativa, que consiste en exámenes con una calificación oficial, requiere restricciones temporales estrictas, medidas de seguridad para evitar el fraude académico y un entorno seguro de ejecución para aislar el código. Además, debería conservar registros auditables de los intentos y resultados para permitir una revisión posterior de la calificación.

La revisión sistemática de Paiva, Leal y Figueira [PLF22] sobre 186 publicaciones entre 2010 y 2020 confirma que la gran mayoría de los sistemas automáticos existentes se han creado para la formación, lo que deja un vacío en la evaluación sumativa mediante exámenes formales de programación. Este hueco es el punto de partida de este TFG.

2.3 La importancia del lenguaje de programación Python

2.3.1 Relevancia actual de Python

La elección de Python como lenguaje básico está motivada por su uso extendido. El índice TIOBE [TIO25] y el *Popularity of Programming Language (PYPL)* [Car25] sitúan a Python entre los lenguajes más populares. Debe señalarse que miden fenómenos distintos, TIOBE combina distintas señales de presencia en la web, mientras que PYPL estima la popularidad a partir de búsquedas de tutoriales en Google. Aun así, ambos índices sitúan a Python entre los lenguajes mejor posicionados en la fecha de consulta, aunque sus valores deben interpretarse de forma prudente al actuar como indicadores metodológicos, no como

cuotas de mercado directas.

El **PYPL**, que analiza las búsquedas de tutoriales en Google al considerarlo un indicador adelantado del interés de aprendizaje, sostiene ese liderazgo estimando en torno al 30 % de la cuota global.

Dado que ambos índices se actualizan periódicamente y utilizan metodologías distintas, sus porcentajes deben interpretarse como indicadores aproximados y dependientes de la fecha de consulta.

En el ámbito educativo, la encuesta anual de Stack Overflow [Sta23] a más de 90 000 desarrolladores en 2023 revela que, entre quienes están aprendiendo a programar, **HTML/CSS** y **JavaScript** aparecen como las tecnologías más usadas, mientras que Python supera claramente a **Structured Query Language (SQL)** dentro de ese grupo.

Python presenta una sintaxis relativamente legible y cercana al pseudocódigo, lo que puede reducir parte de la carga sintáctica inicial de los nuevos estudiantes permitiéndoles centrarse en la lógica de programación sin la dificultad sintáctica de otros lenguajes compilados.

En el contexto universitario español, existen trabajos académicos [Peñ15] que defienden Python como primera aproximación a la programación en titulaciones de ingeniería, por su legibilidad y menor carga sintáctica para estudiantes iniciales.

2.3.2 Retos técnicos específicos de Python para la evaluación automática

Aunque Python es un lenguaje muy conveniente para la enseñanza, automatizar su corrección plantea desafíos técnicos más complejos que los de otros lenguajes. Estas dificultades pueden agruparse en cuatro áreas principales.

En primer lugar, su naturaleza interpretada. Aunque Python suele presentarse como lenguaje interpretado, su implementación habitual compila internamente el código a **bytecode** antes de ejecutarlo. En la práctica docente, los errores relevantes aparecen normalmente durante la ejecución del programa. Esto obliga al sistema evaluador a capturar esos mensajes de error, generalmente técnicos y poco amigables, para transformarlos en una retroalimentación comprensible para un estudiante que está aprendiendo.

En segundo lugar, el tipado dinámico. En Python, las variables no declaran a priori qué tipo de dato almacenarán. Esto hace que sea imposible saber exactamente qué contiene una variable sin ejecutar el programa, lo que limita la utilidad de las herramientas de análisis de código estático que revisan el código sin ejecutarlo.

En tercer lugar, la indentación semántica. A diferencia de la mayoría de lenguajes de programación, Python no usa llaves ni palabras clave para delimitar bloques de código. Como sustituto, se utiliza el nivel de sangría del texto. Un solo espacio mal puesto puede cambiar completamente la lógica del programa. Esto impide nuevamente comparar soluciones mediante búsquedas de texto simples, obligando a trabajar con representaciones estructurales del código. Algunos ejemplos son los árboles de sintaxis abstracta (**AST**), los cuales capturan la lógica del programa con independencia de su apariencia textual.

Por último, la superficie de ataque ampliada. Python permite acceder con facilidad al sistema operativo, archivos del servidor e incluso a sus propios mecanismos internos. Si se ejecuta código de un estudiante directamente en un servidor compartido, existe un riesgo muy cierto de que ese código, de forma ya sea accidental o intencionada, dañe todo el entorno. Por ello, aislar la ejecución mediante un *sandbox* (un entorno aislado, controlado y restringido) se convierte en una decisión arquitectónica fundamental [PLF22].

2.4 Metodología de la revisión narrativa estructurada

Para analizar el estado del arte antes de iniciar en profundidad este proyecto, se ha realizado una selección y análisis de trabajos relacionados siguiendo una revisión narrativa estructurada, inspirada en las directrices de Kitchenham y Charters Kitchenham y Charters [KC07], pero adaptada al alcance temporal y metodológico de un TFG.

La planificación inicial consiste en responder a varias preguntas de investigación. La revisión se organiza en torno a las siguientes siete preguntas de investigación:

1. ¿Qué sistemas automáticos de evaluación de código existen y cuál es su validación?
2. ¿Están diseñados para exámenes síncronos, con restricciones temporales, control de acceso y garantía de la integridad académica?
3. ¿Dónde escriben el código los estudiantes y cómo lo ejecutan?
4. ¿Qué entorno de seguridad usan para aislar código no confiable?
5. ¿Qué tipos de preguntas permiten: completar código, corregir código, razonar trazas o responder test?
6. ¿Qué calidad de retroalimentación pedagógica proporcionan al estudiante y al profesorado?
7. ¿Qué mecanismos de auditoría, registro de eventos o recogida de evidencias contemplan durante la prueba?

Respecto a la segunda fase de ejecución, la búsqueda bibliográfica se llevó a cabo usando las cuatro bases de datos siguientes: [Association for Computing Machinery \(ACM\)](#) Digital Library, [Institute of Electrical and Electronics Engineers \(IEEE\)](#) Xplore, Scopus y Google Scholar para el periodo comprendido entre el año 2000 y 2025. Se priorizó que la fuente describiera un sistema de evaluación automática de código y tuviera una adopción práctica contrastada. Se excluyeron, por tanto, los proyectos basados en experiencias docentes sin explicar detalladamente el sistema usado.

La tercera fase consiste en sintetizar en un informe los resultados por orden cronológico. Se empieza por los trabajos fundacionales y se llega hasta las propuestas más recientes. Se evalúa cada sistema según las variables recogidas en la Tabla 2.1 (sección 2.12). Se elige comparar dichas dimensiones a partir de las preguntas de investigación definidas en el protocolo y el marco de evaluación propuesto por Pieterse [Pie13].

2.5 Evolución histórica de la evaluación automática de código

El problema no es nuevo, ya que cuenta con seis décadas de historia. Hollingsworth [Hol60] describió en 1960 el primer calificador automático de programas en tarjetas perforadas. El avance quedó latente así durante décadas hasta la expansión universitaria de las ciencias de la computación en los años noventa.

Gordon y Kowalski [GK99] propusieron en 1999 una clasificación que sigue siendo útil a día de hoy. Consiste en la comparación de salidas, evaluación basada en tests unitarios, análisis estático y análisis dinámico. Ala-Mutka [Ala05] realizó en 2005 la primera revisión sistemática a gran escala con 32 sistemas, detectando que la comparación de salidas era el mecanismo dominante, que la mayoría de herramientas se habían diseñado para lenguajes

compilados y que la retroalimentación pedagógica era muy escasa. Un lustro después, en 2010, algunos investigadores Ihantola et al. [IAKS10] actualizaron el análisis y lo ampliaron a 62 sistemas. Documentaron una tendencia hacia la integración de **Learning Management System (LMS)**.

La revisión más reciente y exhaustiva considerada en este trabajo es la de Paiva, Leal y Figueira [PLF22]. Es de 2022 y cuenta con 186 publicaciones, la cual confirma tanto la madurez de la evaluación automática como la existencia de brechas persistentes en el soporte para lenguajes interpretados, el contexto de examen y la robustez del *sandboxing*.

2.6 Sistemas académicos clásicos

Los sistemas académicos clásicos presentan aportaciones funcionales relevantes, pero fueron creados para tareas con flujo iterativo, no para exámenes síncronos. Se describen a continuación, y de forma resumida, los cinco sistemas más relevantes para contextualizar las decisiones de diseño de este trabajo.

2.6.1 WebCAT

WebCAT (Web-based Center for Automated Testing) fue desarrollado por Edwards [Edw03] en Virginia Tech. Fue pionero en 2003 por su carácter abierto y su aplicación a la evaluación. El alumno entregaba sus casos de prueba y el sistema corregía el código. Tuvo gran relevancia en educación, pero asumía evaluar repetidamente cada tarea y carecía de un contexto de examen con temporizador, bloqueo de navegación o gestión de sesión.

2.6.2 Marmoset

[SHP+06] desarrollaron en la Universidad de Maryland el concepto de *snapshots*. Era un sistema que hacía capturas periódicas del código mientras se escribía para estudiar longitudinalmente el proceso de escritura de código [SHP+06]. Resultaba útil en proyectos medianos, pero era poco práctico para pequeños ejercicios en exámenes introductorios al no tener soporte nativo para Python ni temporizador.

2.6.3 BOSS

BOSS (BOSS Online Submission and Assessment System) [JGB05] fue uno de los sistemas tempranos de entrega y evaluación automática de código, con soporte para el ciclo de envío, ejecución y evaluación en contexto universitario.

Aunque no se considera una opción actual de implantación para el contexto de este TFG, su valor reside en haber documentado tempranamente los riesgos de ejecutar código de usuario en servidores compartidos y en introducir el concepto de entorno de ejecución aislado (*sandbox*) para mitigarlos.

2.6.4 Submit

Submitty project (Rensselaer Polytechnic Institute) [Sub14] es un sistema de entrega, evaluación automática y apoyo a la corrección docente. Tiene una arquitectura con dos marcos de trabajo desde que fue descrito por primera vez.

Contaba con uno para el *frontend* (Ruby on Rails) y otro (Flask) para la ejecución. Para gestionar envíos concurrentes, administraba la simultaneidad usando RabbitMQ como cola de mensajes y Celery como sistema de tareas distribuidas. Establecía también límites de tiempo de **Central Processing Unit (CPU)** contra los bucles infinitos.

Su principal incompatibilidad con el entorno de examen es que la plataforma estaba pensada para aprender mediante prueba y error. En la versión y configuración revisadas del repositorio GitHub vivo, Submittity está orientado principalmente a entregas iterativas; su adecuación a exámenes síncronos debe evaluarse según la política de intentos, temporización y normalización de salidas configurada en cada curso.

2.6.5 testBench

Fue desarrollada recientemente en la **Universitat Politècnica de Catalunya (UPC)** [Bot25]. Es una herramienta técnicamente sólida que ejecuta código en entornos aislados y muestra los resultados al instante, pero está pensada para tareas asíncronas y sin límite de tiempo, y no se ha identificado un mecanismo específico de supervisión activa para exámenes síncronos.

2.7 Integración con Sistemas de Gestión del Aprendizaje (LMS)

En la última década se ha tendido a integrar los sistemas de evaluación automática dentro de sistemas de gestión del aprendizaje o **Learning Management System (LMS)**. Estas plataformas, como Moodle, gestionan ya la matriculación, plazos y mensajes institucionales. Se busca centralizar la gestión académica con esa integración, pero se depende de la infraestructura del **LMS**.

2.7.1 VPL (Virtual Programming Lab)

Virtual Programming Lab (VPL) es un módulo de Moodle desarrollado en la Universidad de Las Palmas de Gran Canaria [RRH12]. En la literatura española [PRH15] se ha documentado la integración de **VPL** en procesos universitarios de enseñanza de la programación. Deja la ejecución del código a un servidor Linux aislado (jail server) con límites configurables de **Central Processing Unit (CPU)** y memoria.

Aunque permite evaluar código Python, su funcionamiento depende de la configuración del entorno de ejecución y de scripts como `vpl_evaluate.sh`, lo que exige cierta competencia técnica por parte del profesorado. En cuanto al control del examen, **VPL** permite establecer ventanas de disponibilidad y contraseñas de acceso, pero no cuenta con supervisión activa, detección de cambio de pestaña o bloqueo de sesión.

2.7.2 CodeRunner

CodeRunner [LH16] fue desarrollado en la Universidad de Canterbury y funciona como un tipo de pregunta dentro de los cuestionarios de Moodle, por lo que puede aprovechar mecanismos propios del cuestionario, como temporización, aleatorización de las preguntas y control de la cantidad de intentos. Eso precisamente es lo que lo convierte en la opción más cercana al caso de uso de examen dentro de un **LMS** de entre todos los sistemas analizados.

Sí admite Python de forma nativa a través del motor Jobe, permite definir plantillas de evaluación parametrizadas y ofrece un mecanismo de precheck con casos públicos visibles para el alumno. Sus principales limitaciones son la dependencia absoluta de Moodle, la complejidad de configuración si son preguntas avanzadas, la ausencia de supervisión activa (dejando toda la seguridad del examen dependiente de la presencialidad) y una latencia perceptible por los usuarios en sesiones con muchos alumnos si el servidor Jobe no está correctamente dimensionado.

2.8 Soluciones comerciales y jueces en línea

2.8.1 Gradescope

Gradescope [SKGA17] fue presentado como un sistema rápido, flexible y escalable para evaluación académica. Posteriormente, fue adquirido por Turnitin [Tur18]. Su documentación actual describe un módulo de *autograder* basado en contenedores Docker [Graa], así como integración con plataformas docentes como Canvas [Grab] mediante *Learning Tools Interoperability* (LTI).

Presenta, no obstante, tres inconvenientes principales. Son, en primer lugar, su licencia institucional de pago, que resulta inaccesible para algunos departamentos del sistema universitario público español. En segundo lugar, la configuración del autograder, que exige conocimientos avanzados de contenedores. Por último, el uso de plataformas *Software as a Service* (SaaS) externas exige revisar la base jurídica [Eur16], el contrato de encargado de tratamiento, la localización de datos y las posibles transferencias internacionales conforme al *Reglamento General de Protección de Datos* (RGPD).

2.8.2 Jueces en línea

Los jueces en línea son sistemas web automatizados y diseñados tanto para evaluar como calificar el código fuente de programas informáticos enviados por los usuarios. Surgieron a finales de los años noventa para gestionar competiciones algorítmicas, y el primer análisis sistemático de su uso educativo corresponde a Cheang et al. [CKLO03] sobre el UVa Online Judge de la *Universidad de Valladolid* (UVa).

Hoy existen varias plataformas ampliamente utilizadas, como Codeforces [Cod24], LeetCode [Lee24], Kattis [Kat24] o HackerRank [Hac24]. Estas plataformas comparten una arquitectura común donde el usuario envía código, este se ejecuta en un *sandbox*, su salida se compara con la del jurado y se devuelve un veredicto binario (Accepted, Wrong Answer o Time Limit Exceeded). Aunque son útiles para práctica y evaluación algorítmica, varias de estas plataformas no están diseñadas para integrarse directamente con actas universitarias, rúbricas docentes institucionales o revisión administrativa de calificaciones. Su modelo SaaS dificulta el control institucional completo de los datos, y la retroalimentación que ofrecen resulta insuficiente para la enseñanza introductoria [Ala05].

2.9 Retroalimentación pedagógica avanzada y ejercicios de completar código

2.9.1 El problema del «string matching»

La gran mayoría de los sistemas previamente descritos [PLF22] comparan la salida del programa del alumno con la esperada mediante comparación de cadenas de texto («string matching»). Este mecanismo puede penalizar diferencias de formato irrelevantes como, por ejemplo, espacios adicionales, comillas simples frente a dobles o una coma final; y generar falsos negativos cuando la equivalencia funcional no se evalúa semánticamente.

Singh, Gulwani y Solar-Lezama [SGS13] proponen técnicas de generación automática de retroalimentación para programación introductoria, lo que refuerza la conveniencia de ofrecer diagnósticos más informativos que un simple veredicto de salida. Por tanto, resulta conveniente distinguir entre errores de sintaxis, lógica, nombre o tipo, para que un principiante tarde menos en corregir los errores.

2.9.2 Modalidades de pregunta en evaluación introductoria de Python

Los ejercicios de rellenar huecos parten de un esqueleto base de un programa funcional con fragmentos clave eliminados. Esta modalidad está relacionada con *oParsons' Problems* [PH06; WFH21], aunque no es equivalente a ellos. Su ventaja principal es que reduce la carga sintáctica del estudiante y permite centrar la evaluación en estructuras concretas del lenguaje.

No obstante, una evaluación introductoria de programación puede requerir otros formatos complementarios. En primer lugar, los ejercicios de detección y corrección de fallos presentan al alumno un bloque de código Python completo, pero incorrecto, que debe modificar hasta obtener el comportamiento esperado. En segundo lugar, las preguntas de trazas permiten evaluar si el alumno comprende la evolución de variables, condiciones, bucles y llamadas a funciones sin exigir necesariamente la escritura de un programa completo. En tercer lugar, las preguntas tipo test con opciones desplegables permiten comprobar conceptos concretos de sintaxis, semántica o interpretación de resultados con una corrección simple y reproducible.

Por tanto, el prototipo no debe quedar conceptualmente limitado al formato fill-in-the-blanks, aunque este siga siendo el núcleo técnico inicial. El diseño debe permitir asociar cada pregunta a un tipo, a una estrategia de presentación y a un mecanismo de evaluación adecuado. Los mecanismos de evaluación más apropiados son la ejecución de pruebas, comparación de salidas, clave de respuesta, rúbrica docente o revisión posterior.

2.9.3 Evaluación semántica mediante Árboles de Sintaxis Abstracta (AST)

El módulo nativo `ast` de Python permite transformar el código del alumno en un **AST** antes de su ejecución, habilitando la verificación estructural con independencia del formato de salida. Se puede comprobar si el alumno ha usado un bucle `for`, una comprobación de tipos o una llamada recursiva, y no únicamente si el resultado final es correcto.

[Čep24] desarrolló en la Universidad de Twente un prototipo para la corrección automática de exámenes de programación en Python que combina pruebas de salida y análisis de código fuente basado en **Abstract Syntax Tree (AST)**. El trabajo mostró una correlación positiva con la evaluación manual, aunque también señaló la necesidad de refinar el sistema antes de sustituir por completo la revisión humana.

Herramientas complementarias como Pylint [PyC24] permiten realizar análisis estático del código, detectar errores, comprobar convenciones de estilo e identificar posibles problemas de mantenibilidad que los tests dinámicos pueden no capturar.

2.10 El rol y las limitaciones de los modelos de lenguaje (LLM)

Los modelos de lenguaje de gran tamaño (LLM), como GPT-4, son capaces de detectar errores en el código de un estudiante y de generar explicaciones útiles para la retroalimentación formativa [PVM+23].

Sin embargo, su uso como mecanismo directo de calificación plantea problemas de reproducibilidad, trazabilidad y consistencia, especialmente si la respuesta del modelo varía entre ejecuciones [DKG23; KCM+23]. También existe el riesgo de que el modelo genere explicaciones incorrectas o ignore fallos reales. Por último, enviar trabajos de estudiantes a servicios externos introduce problemas de privacidad, dependencia de terceros y latencia difícil de controlar en sesiones con muchos alumnos.

Por ello, el sistema que califica un examen universitario debe ser algorítmico, determinista y auditable. La **Inteligencia Artificial (IA) Generativa** puede ser una tecnología complementaria útil para generar retroalimentación formativa, resumir errores frecuentes o producir informes pedagógicos anonimizados para el estudiante y el profesorado. Sin embargo, en el alcance de este TFG no debe emplearse como sustituto directo del motor de evaluación ni como mecanismo autónomo para imponer penalizaciones académicas. Su salida debe considerarse auxiliar, revisable y no vinculante.

2.11 Seguridad durante la ejecución e integridad académica

2.11.1 Sandboxing y protección del servidor

Cuando un estudiante envía código Python para ser ejecutado, el sistema debe tratarlo como código no confiable [CD13]. La ejecución de código no confiable puede provocar consumo abusivo de recursos, acceso indebido al sistema de archivos o interacción no deseada con servicios del servidor [PLF22].

Restringir la ejecución de código de terceros utilizando únicamente los mecanismos internos del propio lenguaje Python no es una medida de seguridad efectiva. Las estrategias que intentan confinar el código a nivel de aplicación, como alterar el entorno de ejecución o restringir el acceso a las funciones nativas del lenguaje (*builtins*), presentan vulnerabilidades inherentes.

Esto se debe a la naturaleza dinámica de Python, la cual permite la introspección. Es una característica por la cual el código puede analizar y explorar su propia jerarquía de clases en memoria. Gracias a ello, los profesionales [Mar21] indican que un usuario con conocimientos avanzados podría evadir las barreras impuestas desde el propio código y recuperar el acceso a funciones restringidas. Por ello, con el fin de garantizar una evaluación segura, resulta estrictamente necesario aplicar un aislamiento externo al lenguaje.

La solución real pasa por aislar la ejecución a nivel del sistema operativo mediante tres enfoques:

El primero es usar contenedores (Docker) que encapsulan el proceso del estudiante y limitan su acceso a **Central Processing Unit (CPU)**, memoria y red [Doc24]. Estos deben configurarse para evitar fugas hacia el sistema anfitrión.

El segundo es el uso de controles **Mandatory Access Control (MAC)** (AppArmor, SELinux), los cuales actúan como una segunda capa que restringe a qué recursos del sistema puede acceder ese proceso. Plataformas como Open edX [Ope24] recomiendan reforzar el aislamiento con mecanismos del sistema operativo, como AppArmor, cuando se ejecuta código no confiable.

El tercer enfoque consiste en emplear mecanismos de aislamiento más fuertes que un contenedor estándar. Entre ellos se encuentran las microVMs, como Firecracker [ABI+20]; los *sandboxes* de contenedores basados en kernel de usuario, como gVisor [gVi]; y herramientas de aislamiento de procesos basadas en namespaces, cgroups y filtros seccomp-bpf, como nsjail [nsj].

2.11.2 Supervisión del entorno del cliente

Asegurar el servidor no es suficiente si el estudiante puede abrir otra pestaña del navegador para buscar respuestas o usar una **IA** que le resuelva los problemas lógicos. Los lockdown browsers, como Safe Exam Browser o Respondus, bloquean el equipo durante el examen, son invasivos al deshabilitar atajos de teclado o forzar la pantalla completa, y pueden dar problemas de compatibilidad con ordenadores personales.

Una estrategia menos invasiva consiste en registrar eventos de comportamiento del navegador, como `visibilitychange` y `blur`, para generar trazas de auditoría mediante JavaScript [MDN26a; MDN26b]. A partir de esos eventos, y solo cuando exista configuración institucional, información previa suficiente, base jurídica adecuada y autorización técnica del navegador, puede plantearse un módulo de recogida de evidencias que active la grabación de pantalla, audio de micrófono y cámara frontal durante una ventana temporal acotada. Su finalidad no es penalizar automáticamente, sino conservar material revisable por el profesorado cuando ya se ha producido un evento relevante de pérdida de foco o salida de pestaña.

La literatura [HLB10] sobre evaluación en línea respalda que el diseño de la evaluación y la supervisión pueden reducir el riesgo de fraude, aunque no garantizan integridad absoluta del examen. Deben entenderse como medidas disuasorias y complementarias, sometidas a proporcionalidad, minimización de datos, limitación temporal y revisión humana posterior.

2.11.3 Cumplimiento con el RGPD

Cualquier sistema que gestione identidades, entregas, respuestas, calificaciones, eventos de auditoría o evidencias audiovisuales maneja información personal identificable (**PII**) y, por tanto, está sujeto tanto al **Reglamento General de Protección de Datos (RGPD)** como a la **Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales (LOPDGDD)** española [Eur16; 18]. Las plataformas que almacenan datos en servidores fuera de la Unión Europea (como Gradescope o HackerRank) pueden generar dificultades adicionales de cumplimiento normativo, especialmente si implican transferencias internacionales de datos o tratamiento por encargados externos.

La incorporación de grabación de pantalla, audio y cámara frontal aumenta de forma significativa la intrusión del tratamiento. Por ello, este tipo de evidencia debe configurarse como un módulo excepcional y proporcionado, no como una grabación general e indiscriminada de todo el examen. Su activación debe limitarse a eventos concretos de auditoría, como pérdida de foco o salida de pestaña, y su conservación debe estar temporalmente acotada. Además, debe informarse al alumno antes de iniciar la prueba sobre qué datos se

recogen, con qué finalidad, durante cuánto tiempo, quién puede acceder y cómo se pueden ejercer los derechos reconocidos por la normativa.

No deben utilizarse documentos oficiales de identidad (**DNI**) como identificadores visibles ni como campos operativos ordinarios. Si se requiere un identificador estable, debe emplearse preferentemente un identificador académico o correo institucional. Para usos analíticos o generación de informes mediante modelos de lenguaje, los datos deben anonimizarse o, cuando la anonimización completa no sea posible, seudonimizarse, evitando enviar a servicios externos nombres, correos, identificadores académicos, grabaciones o fragmentos que permitan identificar directamente al alumno.

El modelo de datos deberá separar la información académica estrictamente necesaria de los artefactos de auditoría. Las respuestas del alumno, tanto correctas como incorrectas, se conservarán para justificar la calificación y permitir revisión posterior. Las evidencias audiovisuales, si se activan, deberán tener un periodo de conservación más restrictivo y acceso limitado al profesorado autorizado o a los responsables institucionales correspondientes. La generación de informes con **LLM** se realizará únicamente sobre datos minimizados y no se empleará para modificar calificaciones oficiales.

2.12 Análisis comparativo del estado del arte

En la Tabla 2.1 se sintetizan los sistemas analizados en las secciones anteriores según las dimensiones más relevantes para este proyecto. Cada dimensión se interpreta del siguiente modo:

1. **Python nativo** indica soporte directo para Python sin configuración manual avanzada.
2. **Contexto examen** refleja la existencia de mecanismos de temporización, control de acceso y gestión de sesión.
3. **Retroalimentación** se valora en una escala de 1 a 4 (1 = veredicto binario; 2 = diferencia de salidas; 3 = análisis de errores categorizado; 4 = retroalimentación narrativa o pedagógica).
4. **Integración LMS** indica compatibilidad nativa con Moodle u otro **Learning Management System (LMS)** institucional.
5. **Sandbox documentado** señala la existencia de un mecanismo de aislamiento con límites de recursos explicitado en la documentación.
6. **Libre** indica disponibilidad como software de código abierto.
7. **Complejidad docente** refleja el nivel técnico requerido para crear y gestionar evaluaciones.

El patrón obtenido del análisis es que ninguno de los sistemas estudiados cubre simultáneamente todas las dimensiones críticas para el caso de uso objetivo. La descripción breve de cada uno es la siguiente:

Los sistemas académicos clásicos son buenos en pedagogía, pero carecen de soporte para el contexto de examen.

Los *plugins* de **Learning Management System (LMS)** son los más cercanos en integración institucional, aunque dependen de la infraestructura del **LMS** y la configuración por parte del docente puede ser compleja. Su principal limitación no es la ausencia de soporte

Tabla 2.1: Análisis comparativo de sistemas de evaluación automática de código.

Sistema / Plataforma	Python nativo	Contexto examen	Retroal. (1–4)	LMS	sandbox	Libre	Complejidad
WebCAT	Parcial	No	3	No	Parcial	Sí	Alta
Marmoset	No	No	2	No	Sí	No	Alta
Submit	Parcial	No	2	No	Parcial	No	Alta
BOSS	No	No	1	No	Sí	No	Alta
VPL (Moodle)	Configurable	Parcial	2	Sí	Sí	Sí	Alta
CodeRunner	Sí	Parcial	3	Sí	Sí	Sí	Media
Gradescope	Sí	Parcial	2	Sí	Sí	No	Alta
HackerRank	Sí	Parcial	2	No	Sí	No	Media
Kattis / UVa	Sí	No	1	No	Sí	No	Baja
LeetCode	Sí	No	1	No	Sí	No	Baja
testBench	Sí	No	2	No	Sí	Sí	Media
Prototipo propuesto	Previsto en diseño	Previsto	3 (prevista)	No previsto	Previsto	Previsto	Baja prevista

Fuente: elaboración propia a partir de la revisión narrativa estructurada de la literatura

para Python, sino la dependencia de Moodle y la complejidad de configuración en escenarios de examen.

Los sistemas comerciales ofrecen una mejor **User Experience (UX)**, pero tienen un coste potencialmente prohibitivo y el alojamiento externo es incompatible con los requisitos de autonomía y soberanía de datos del proyecto.

Para terminar la comparativa, los jueces en línea priorizan la eficiencia algorítmica, lo que los hace poco adecuados para la enseñanza introductoria universitaria.

2.13 Lagunas identificadas y justificación del proyecto

Del análisis sistemático presentado en las secciones anteriores se obtienen las cinco lagunas del estado del arte que justifican este **TFG**.

Laguna 1. Dentro del alcance de la revisión realizada, no se ha identificado una solución autónoma que combine exámenes síncronos de Python, ejecución aislada, trazabilidad y supervisión ligera del cliente sin depender de un **LMS**. Los sistemas existentes parten del paradigma de la tarea, por lo que cuentan con plazos flexibles y múltiples intentos. Además, han ido añadiendo funcionalidades de examen solo de forma parcial. Un examen impone requisitos concretos tales como el control del tiempo de inicio y fin, envío único e irrevocable, entorno aislado y actas verificables [**PLF22**].

Laguna 2. La segunda laguna identificada es la alta complejidad para el docente. La literatura [**IAKS10**] identifica una carga técnica relevante en la configuración de sistemas de evaluación automática; en herramientas actuales, esta carga puede incluir scripts de evaluación, servidores de ejecución o contenedores, según la plataforma.

Laguna 3. Hay una retroalimentación insuficiente para Python. Los sistemas con mejor retroalimentación están optimizados para lenguajes compilados de tipo estático como C++. La corrección mediante análisis del **Abstract Syntax Tree (AST)**, que comprueba si el estudiante usó las estructuras exigidas al margen de si el resultado es correcto, apenas está presente en las herramientas actuales [**Čep24**] aunque podría reducir determinados falsos negativos asociados a diferencias puramente textuales; esta hipótesis debe validarse empíricamente en el sistema propuesto.

Laguna 4. Existe una dependencia de infraestructura o de pago. Los sistemas más completos requieren plataformas institucionales como Moodle o Active Directory. Además, la compra de licencias comerciales supone una barrera de entrada inasumible para muchas universidades. La solución debe poder desplegarse de forma autónoma y gratuita.

Laguna 5. No se ha identificado, dentro del alcance de esta revisión, una solución sencilla que combine aislamiento robusto del servidor con registro ligero de eventos del navegador para fines de auditoría. Los sistemas con *sandbox* robusto no vigilan el navegador del estudiante, y viceversa.

En definitiva, existe un espacio no cubierto que justifica el diseño de un prototipo funcional de aplicación web orientada a integrar varias características necesarias. Se requiere autoría de exámenes Python, soporte inicial de varios tipos de preguntas, accesibilidad para docentes sin perfil técnico, corrección automática basada en ejecución o comparación determinista cuando proceda, almacenamiento completo de respuestas correctas e incorrectas, *sandbox* documentado y razonablemente restringido, temporizador estricto, registro de eventos del navegador, posible recogida proporcionada de evidencias asociadas a salidas de pestaña y generación auxiliar de informes pedagógicos anonimizados mediante LLM. Todo ello debe desplegarse de forma autónoma, auditable y compatible con los principios de protección de datos desde el diseño.

Capítulo 3

Objetivos

El objetivo principal de este **Trabajo Fin de Grado (TFG)** es el diseño e implementación de una herramienta software en forma de aplicación web para la evaluación automatizada, segura y auditable de ejercicios y preguntas de programación en Python.

Está destinada a estudiantes sin experiencia previa en la asignatura de Informática y pretende aumentar la eficiencia del proceso docente, mejorar la experiencia de aprendizaje del alumnado y facilitar al profesorado la revisión posterior de resultados e incidencias. Para estructurar el desarrollo de forma coherente y auditable, el proyecto tiene los siguientes cinco objetivos específicos:

1. Desarrollar la arquitectura *full-stack* de la plataforma web.
2. Diseñar e implementar un modelo de datos relacional para exámenes, tipos de preguntas, respuestas, entregas, resultados, eventos de auditoría, evidencias asociadas e informes de retroalimentación.
3. Construir un motor de evaluación automatizada y segura para preguntas de Python, incluyendo ejecución aislada de código cuando sea necesario.
4. Diseñar la interfaz de examen del alumnado y un panel básico de resultados para el profesorado, con control de tiempo, soporte de varios formatos de pregunta y registro de eventos de integridad académica.
5. Validar el sistema integrado mediante pruebas funcionales, de seguridad, de persistencia, de auditoría y de usabilidad.

Todos los objetivos se han formulado siguiendo el criterio SMART (*Specific, Measurable, Achievable, Relevant, Time-bound*) y se han acompañado de criterios de cumplimiento verificables, próximos al concepto de **Definition of Done (DoD)**.

El desarrollo detallado de todos los objetivos, incluyendo sus límites y justificación, es el siguiente:

Objetivo 1. Desarrollar arquitectura *full-stack* de la aplicación web.

Este objetivo incluye la creación del esqueleto completo del sistema. Su consecución es un requisito previo para el desarrollo del resto de componentes.

El desarrollo comenzará por el servidor *backend* que gestiona peticiones **HyperText Transfer Protocol (HTTP)**. Posteriormente se implementará una **User Interface (UI)** que

permita al alumno identificarse mediante nombre, apellidos e identificador académico o correo institucional, sin utilizar documentos oficiales de identidad como identificador operativo ordinario ni en texto plano, y acceder al examen correspondiente.

Decisiones tecnológicas y límites.

El *backend* se implementa con *FastAPI*, el cual se adopta por su integración con Python, su soporte para APIs tipadas y su capacidad para definir endpoints asíncronos cuando resulte necesario. El *frontend* se crea usando *HyperText Markup Language (HTML)*, *Cascading Style Sheets (CSS)* y JavaScript sin *frameworks* de cliente adicionales. Esto evita añadir un framework de cliente adicional y reduce la complejidad del frontend.

Criterios de cumplimiento.

El objetivo se considerará cumplido si:

- (a) el servidor arranca sin errores y responde a peticiones en la ruta raíz.
- (b) el formulario de identificación acepta, valida y procesa correctamente las entradas de nombre, apellidos e identificador académico.
- (c) la navegación al cuestionario se produce sin errores de consola en los navegadores de uso mayoritario Chrome, Firefox y Edge.

Planificación.

Como es el fundamento de todo, este objetivo se aborda en la primera fase del desarrollo práctico, con una duración estimada de cinco semanas.

Objetivo 2. Implementar un sistema de persistencia de datos relacional.

Este objetivo consiste en diseñar e integrar correctamente la parte de almacenamiento que mantiene al resto del sistema, una capa de persistencia relacional escalable. La base de datos debe organizar varias entidades principales de forma paralela. Se trata de los exámenes, banco de preguntas, tipos de pregunta, opciones de respuesta, soluciones de referencia, casos de prueba, rúbricas simples, datos identificativos mínimos de los alumnos, respuestas enviadas, calificaciones, errores de ejecución, salidas obtenidas y eventos de auditoría.

El modelo deberá permitir registrar respuestas correctas e incorrectas, no únicamente la calificación final. Para ello, cada entrega almacenará el contenido enviado por el alumno, el tipo de pregunta, el resultado de corrección, la puntuación obtenida, los mensajes de error relevantes y el estado de revisión. En el caso de preguntas de trazas o respuestas en lenguaje natural, el sistema deberá conservar la respuesta textual para revisión docente o retroalimentación posterior. En el caso de preguntas tipo test, deberá registrar la opción seleccionada y la opción correcta definida por el profesorado.

Además de las entidades principales, el modelo incluirá una tabla de auditoría (*AuditEvent*) para registrar eventos relevantes de integridad académica, como pérdida de visibilidad o foco de la pestaña. Como ampliación condicionada al cumplimiento legal y técnico, podrá incluir una entidad separada para evidencias (*EvidenceArtifact*), donde se referencien de forma controlada grabaciones de pantalla, audio o cámara asociadas a un evento concreto. Finalmente, se contempla una entidad de informes de retroalimentación (*FeedbackReport*), generados de forma anonimizada o seudonimizada, tanto para el alumno como para el profesorado.

Decisiones tecnológicas y límites.

El almacenamiento se gestiona mediante el *Object-Relational Mapping (ORM)* llama-

do SQLAlchemy. Para el entorno de desarrollo local se utilizará SQLite por su ligereza. Para despliegues con concurrencia real en producción se mantendrá compatibilidad con PostgreSQL como opción recomendada, sin impedir una demo local reproducible con SQLite.

Criterios de cumplimiento.

El objetivo se verificará mediante una batería de pruebas de integración que comprueben todos los aspectos. Eso incluye la inclusión correcta de preguntas y soluciones de referencia, el registro de respuestas de alumnos y la recuperación consistente de calificaciones, sin pérdida ni corrupción de datos.

Planificación.

Como la base de datos es necesaria para el resto de componentes, el esquema se diseñará en paralelo al Objetivo 1. Se mejorará de forma incremental conforme avancen los objetivos siguientes con el propósito de tenerla actualizada.

Objetivo 3. Construir un motor de ejecución automatizada y segura (*sandbox*) para código Python.

Este objetivo incluye el núcleo técnico del proyecto. Se trata de crear un motor de evaluación capaz de procesar varios tipos de preguntas de Python. En los ejercicios de rellenar huecos, el sistema recibe los fragmentos completados por el alumno, reconstruye el programa y lo evalúa mediante casos de prueba. En los ejercicios de detección y corrección de fallos, el alumno puede modificar un bloque de código completo y el sistema compara su comportamiento con la salida esperada o con tests funcionales definidos por el profesorado. En las preguntas tipo test, el motor compara la opción seleccionada con la clave de respuesta. En las preguntas de trazas, el sistema conserva la explicación breve del alumno y permite revisión docente o retroalimentación auxiliar, evitando que la calificación oficial dependa de un LLM.

La corrección no se realizará mediante comparación sintáctica de cadenas de texto, lo cual puede generar falsos negativos cuando existen diferencias textuales irrelevantes; esta limitación se mitigará mediante ejecución de pruebas funcionales.

Decisiones tecnológicas y límites.

La mayor dificultad de este objetivo es la seguridad. Permitir que una aplicación web ejecute código arbitrario enviado por un usuario amenaza la integridad del servidor. Se pueden provocar denegaciones de servicio (por ejemplo, con bucles infinitos), intentar leer o modificar recursos del sistema o inyectar cargas maliciosas.

Para mitigar este riesgo crítico, se garantizará la ejecución segura del código del alumno mediante un entorno aislado real a nivel de sistema operativo utilizando contenedores Docker, evitando depender de soluciones basadas únicamente en subprocess (Python puro) en producción. La ejecución del estudiante se delegará a contenedores Docker efímeros, creados bajo una política restrictiva por defecto, sin acceso a Internet, con el sistema de archivos en modo solo lectura y con límites estrictos de Central Processing Unit (CPU) y memoria gestionados por cgroups. Además, se aplicarán tiempos de espera cortos (configurables en el rango de 2–3 segundos) para finalizar ejecuciones no acotadas y mitigar ataques basados en bucles infinitos.

De forma complementaria, y siempre que la infraestructura lo permita, se reforzará el aislamiento mediante perfiles de seguridad (por ejemplo, seccomp/AppArmor), ejecución sin privilegios y eliminación de capacidades innecesarias del contenedor. El objetivo no es construir un *entorno invulnerable*, sino una capa de protección robusta, auditable y pro-

porcionada al caso de uso, que reduzca drásticamente la superficie de ataque y limite el impacto de entradas maliciosas.

Criterios de cumplimiento.

El objetivo se medirá mediante una batería de al menos veinte casos de prueba que cubra todos los perfiles. El sistema debe gestionar correctamente:

- (a) código correcto: la calificación debe ser máxima y la salida debe coincidir con la referencia.
- (b) código con errores lógicos: la calificación no puede ser máxima y, en ningún caso, debe comprometer la estabilidad del servidor.
- (c) código con bucle infinito o ejecución no acotada: el proceso debe finalizarse por *timeout* dentro del límite configurado (2 s a 3 s), liberando recursos de forma inmediata.
- (d) fragmentos que intenten importar módulos restringidos, acceder al sistema de archivos, invocar procesos o realizar operaciones potencialmente peligrosas; el contenedor debe impedir estas acciones y el servidor no debe verse comprometido.

El sistema deberá bloquear, contener o finalizar correctamente todos los casos maliciosos incluidos en la batería de pruebas definida para la validación, sin fugas de información observadas ni degradación apreciable bajo las condiciones de validación especificadas.

Planificación.

Este objetivo se planifica para la segunda fase del desarrollo, una vez que la arquitectura base y la capa de datos estén operativas, con una duración estimada de cuatro semanas.

Objetivo 4. Diseñar una interfaz de examen con temporizador, monitorización de actividad y panel de resultados para el profesor.

Este objetivo cubre la experiencia del alumno durante la prueba. La vista de examen debe presentar distintos tipos de pregunta, incluyendo bloques de código con huecos editables, bloques completos de código modificables para corregir fallos, preguntas de trazas con un campo de respuesta breve en lenguaje natural y preguntas tipo test con opciones desplegables. La interfaz docente incluirá un panel básico de resultados que permita consultar entregas, calificaciones, respuestas correctas e incorrectas, estado de corrección, errores de ejecución, respuestas textuales y eventos de auditoría asociados a cada intento.

También deberá mostrar un temporizador descendente visible con una barra de progreso e implementar un mecanismo que detecte el abandono de la pestaña activa del navegador. Si el alumno cambia de pestaña, pierde el foco o minimiza la ventana, el sistema registrará el evento con fines de auditoría, sin aplicar penalización automática. Cuando la configuración institucional y los permisos técnicos lo permitan, el evento podrá activar la recogida de evidencias audiovisuales acotadas (pantalla, micrófono y cámara frontal) para revisión posterior por el profesorado.

Límites.

El límite principal viene impuesto por la arquitectura de seguridad de los navegadores web modernos. Una página web no puede tomar control total del dispositivo del usuario sin instalar software adicional ni puede activar de forma silenciosa la captura de pantalla, cámara o micrófono. Por ello, la monitorización se limitará al registro de eventos de pérdida de foco mediante `visibilitychange` y `blur`, y cualquier recogida de evidencias audiovisuales dependerá de permisos explícitos del navegador, configuración previa, información

transparente al alumno y cumplimiento normativo.

No se pretende conseguir la funcionalidad de un lockdown browser completo como Safe Exam Browser. Esta decisión de diseño es deliberada: se opta por una combinación proporcional de temporización, registro de eventos de actividad, posible evidencia acotada y revisión posterior por el profesorado.

El entorno seguro de evaluación no dependerá únicamente de las restricciones del navegador o del cliente. Para garantizar la integridad del tiempo de examen, el servidor será la única fuente de verdad. El temporizador mostrado en el *frontend* actuará únicamente como **User Experience (UX)**, mientras que el servidor validará matemáticamente la *hora_inicio* y la duración máxima permitida.

Además, aplicando el principio de Privacy by Design, la **User Interface (UI)** incorporará información clara sobre el tratamiento de datos personales, minimización de datos, finalidad de las evidencias, periodo de conservación y derechos del alumno conforme al **Reglamento General de Protección de Datos (RGPD)** antes de acceder a la prueba.

Criterios de cumplimiento.

El objetivo se verificará comprobando que:

- (a) el sistema finaliza la prueba al alcanzarse la duración máxima permitida definida en servidor y registra la entrega de forma automática, con un margen acotado y medido en las pruebas, preferiblemente inferior a unos pocos segundos.
- (b) el evento de pérdida de visibilidad o foco deberá registrarse en los navegadores definidos en el plan de pruebas bajo los escenarios especificados. Quedará disponible para revisión manual por el profesorado, sin modificar por sí solo la calificación.
- (c) los campos de código son los únicos elementos editables del examen.
- (d) el profesorado puede consultar, al menos por examen y alumno, la calificación obtenida, el estado de la entrega y los eventos de auditoría registrados. Tendrá un panel básico de resultados.

Planificación.

Este objetivo se desarrolla en la tercera fase del proyecto, cuando el motor de corrección esté operativo. Tiene una duración estimada de cuatro semanas.

Objetivo 5. Validar el sistema integrado mediante pruebas funcionales y de usabilidad.

Una vez integrados los cuatro componentes anteriores, este objetivo somete el sistema completo a validación mediante varias líneas complementarias de validación.

La primera línea de validación comprueba la funcionalidad y robustez del flujo de datos mediante pruebas de extremo a extremo sobre el flujo completo de datos usando objetos de transferencia de datos (**DTO**).

Esta línea ejecutará de forma automatizada el proceso completo. En orden temporal, va la identificación del alumno, carga del examen desde la base de datos al *frontend*, realización de la prueba con distintos tipos de pregunta, envío de respuestas correctas e incorrectas, recepción de la calificación automática cuando proceda y el almacenamiento completo de la entrega. Se validará que el sistema gestiona correctamente ejercicios de rellenar huecos, corrección de fallos en código editable, preguntas de trazas y preguntas tipo test.

Además, se validará explícitamente la tolerancia a fallos del sistema y su comportamiento ante reintentos, garantizando el principio de *idempotencia*: fallos de red o reenvíos

por parte del alumno no deben duplicar entregas ni alterar la calificación registrada.

La segunda línea es empírica y evalúa la usabilidad del sistema con usuarios reales, verificando que la herramienta es percibida como útil, comprensible y accesible por el perfil de estudiante para el que fue diseñada.

Por último, se validará el rendimiento bajo concurrencia mediante pruebas de carga y estrés, asegurando tiempos de respuesta acotados y ausencia de caídas cuando múltiples alumnos utilicen la plataforma simultáneamente.

Límites.

Como límite intrínseco, la validación del sistema integrado depende del correcto funcionamiento acumulado de los componentes anteriores. Las pruebas de integración no pueden realizarse si los componentes están incompletos, por lo que se reserva para la fase final del proyecto.

Como límite extrínseco, el tamaño de la muestra de usuarios estará sujeto a la disponibilidad voluntaria de participantes. Una muestra reducida puede aportar evidencia orientativa para detectar problemas de usabilidad en un prototipo, pero no permite realizar inferencias estadísticas robustas sobre la población completa [Nie00].

Criterios de cumplimiento.

- **Pruebas funcionales e idempotencia:**

- el flujo completo de identificación, examen, almacenamiento de respuestas y calificación debe ejecutarse sin errores en al menos 10 simulaciones automatizadas con perfiles de respuesta variados. Las respuestas recibidas podrán ser código correcto, código incorrecto, respuesta vacía, código malicioso, opción tipo test correcta, opción tipo test incorrecta y respuesta breve de traza.
- la calificación debe coincidir con la esperada en el 100 % de los casos en los que exista una clave de corrección determinista.
- las respuestas que no puedan calificarse de forma plenamente automática deberán quedar registradas con estado de revisión pendiente o revisada, sin pérdida de información.

- **Pruebas de rendimiento y carga (estrés):**

- Las pruebas funcionales se integrarán en **Continuous Integration (CI)**; las pruebas de carga se ejecutarán como validación específica documentada fuera de la canalización **CI** ordinaria.
- se realizarán pruebas de carga con una concurrencia objetivo de 100 usuarios simultáneos, registrando la latencia p_{95} total, la latencia p_{95} del *sandbox* si interviene ejecución de código, la tasa de errores y las condiciones de la prueba. El cumplimiento se considerará alcanzado si el sistema mantiene la corrección de las calificaciones, no pierde entregas y conserva tiempos de respuesta compatibles con el uso docente previsto.

- **Pruebas de usabilidad:** la usabilidad se evaluará mediante **System Usability Scale (SUS)**; se considerará indicio favorable obtener una puntuación media igual o superior a 70, indicando siempre tamaño muestral, desviación y limitaciones de inferencia. Es el umbral que la literatura [BKM09; Bro96] establece como indicativo de una usabilidad aceptable.

Planificación.

Este objetivo se planifica para realizarse en la recta final del proyecto, de forma coincidente con la redacción de los capítulos de resultados y conclusiones. Tiene una duración estimada de cuatro semanas.

Capítulo 4

Análisis de alternativas y decisiones de diseño

4.1 Criterios de decisión

4.2 Modelo de persistencia de datos

4.2.1 Alternativas de organización (una o varias bases de datos)

4.2.2 Alternativas de motor (SQLite, PostgreSQL, NoSQL)

4.2.3 Decisión tomada y su justificación

Se adopta una única base de datos relacional con tablas diferenciadas para exámenes, preguntas, respuestas, entregas, resultados, eventos de auditoría, evidencias e informes. La decisión permite mantener integridad referencial y trazabilidad entre la definición de una pregunta, la respuesta del alumno y la calificación obtenida.

El diseño incorpora un campo `question_type` para distinguir entre preguntas de rellenar huecos, corrección de fallos, trazas y tipo test. Esta decisión evita crear un sistema rígido asociado a un único formato. Los atributos comunes se mantienen en la tabla `Question`, mientras que los datos específicos de cada modalidad se almacenan en tablas auxiliares o campos estructurados: plantillas de código, casos de prueba, salidas esperadas, opciones de respuesta, solución correcta y rúbrica simple.

Las respuestas se almacenan siempre, con independencia de que sean correctas o incorrectas. Esta decisión es necesaria para justificar la calificación, permitir revisión posterior, analizar errores frecuentes y generar informes de retroalimentación anonimizados. Las evidencias de auditoría, cuando existan, se separan del núcleo académico para aplicar controles de acceso y periodos de conservación más restrictivos.

4.3 Motor de ejecución segura de código (sandbox)

4.3.1 Ejecución sin aislamiento

4.3.2 Aislamiento a nivel de aplicación (subprocess + rlimit)

4.3.3 Aislamiento a nivel de sistema operativo (Docker Engine o Docker Desktop)

4.3.4 Alternativas avanzadas (gVisor, Firecracker, nsjail, WebAssembly)

4.3.5 Decisión tomada y su justificación

4.4 Validación estática previa del código del alumno

4.4.1 Filtro previo a la ejecución

4.4.2 Alternativas

4.4.3 Decisión tomada

4.5 Modalidad de acceso y entorno de examen

4.5.1 Aplicación web o instalable

4.5.2 Lockdown browsers (Safe Exam Browser, Respondus)

4.5.3 Arranque restringido por hardware (pendrive/Live USB)

4.5.4 Modificación de BIOS/UEFI

4.5.5 Supervisión ligera en cliente y control efectivo en servidor

Se adopta una estrategia de supervisión ligera en cliente combinada con control efectivo en servidor. El cliente detecta eventos como pérdida de foco, cambio de pestaña o minimización de ventana. El servidor registra dichos eventos con marca temporal, identificador del intento y contexto mínimo necesario.

Como ampliación, el sistema puede asociar evidencias audiovisuales a eventos concretos. Esta opción no debe entenderse como vigilancia continua por defecto, sino como una recogida limitada y condicionada a permisos técnicos, información previa, base jurídica adecuada, proporcionalidad y configuración institucional. La evidencia no produce penalización automática; queda disponible para revisión humana posterior.

Esta alternativa se considera más proporcionada que un lockdown browser completo y más realista que confiar exclusivamente en restricciones del navegador. La integridad principal de la prueba sigue dependiendo del *backend*, control del tiempo, unicidad del intento, validación de entregas, corrección reproducible y registro persistente.

4.6 Resumen de decisiones y trazabilidad con los objetivos

Capítulo 5

Desarrollo del TFG

5.1 Metodología de trabajo

5.1.1 Enfoque metodológico

5.2 Planificación y Fases del Proyecto

La planificación se ajusta para contemplar desde el inicio varios tipos de preguntas.

En la Fase 1 se diseña la arquitectura general y el modelo de datos, incluyendo el campo de tipo de pregunta, las respuestas del alumno, los eventos de auditoría, las evidencias y los informes de retroalimentación.

En la Fase 2 se implementa el motor de evaluación, priorizando los ejercicios de rellenar huecos y corrección de fallos mediante ejecución aislada, y añadiendo corrección determinista para preguntas tipo test.

En la Fase 3 se desarrolla la interfaz multiformato del examen y el panel docente, junto con el registro de eventos de pérdida de foco y la posible recogida acotada de evidencias.

En la Fase 4 se validan los flujos completos, la persistencia de respuestas correctas e incorrectas, robustez del *sandbox*, usabilidad y la generación de informes anonimizados.

5.2.1 Fase 1: Arquitectura y Modelo de Datos

5.2.2 Fase 2: Motor de Evaluación y Sandbox

5.2.3 Fase 3: Interfaz y Auditoría

5.2.4 Fase 4: Validación y Despliegue

5.3 Ejecución del Proyecto y Desviaciones

5.3.1 Problemas encontrados en el aislamiento de código

5.3.2 Desviaciones en la persistencia y concurrencia

5.3.3 Retos en el frontend sin frameworks

Capítulo 6

Resultados y discusión

6.1 Resultado funcional global

6.1.1 Descripción del prototipo final

6.1.2 Flujo de valor para el docente y el alumno

6.2 Validación del sandbox y la seguridad

6.2.1 Eficacia del análisis estático (AST)

6.2.2 Aislamiento a nivel de Sistema Operativo

6.3 Pruebas de rendimiento y concurrencia

6.3.1 Metodología de la prueba de carga

6.3.2 Resultados de latencia y throughput

6.3.3 Discusión sobre el cuello de botella de base de datos

6.4 Contribuciones originales y metodológicas

6.4.1 Motor de corrección «Batch» determinista

6.4.2 Privacidad por diseño (Privacy by Design)

6.4.3 Auditoría de integridad académica no invasiva

6.5 Resultados negativos y limitaciones técnicas

6.5.1 Insuficiencia del «Subprocess» local para entornos compartidos

6.5.2 Limitaciones del soporte inicial multiformato

Aunque el prototipo contempla varios tipos de pregunta en el modelo conceptual y en la estructura de datos, el grado de automatización no es idéntico en todos los formatos. Los

ejercicios de rellenar huecos y corrección de fallos son los más adecuados para corrección automática mediante ejecución de código y comparación con tests. Las preguntas tipo test presentan una corrección directa mediante clave de respuesta.

En cambio, las preguntas de trazas en lenguaje natural requieren mayor cautela, ya que una respuesta breve puede ser parcialmente correcta, ambigua o expresada con terminología distinta a la esperada. Por ello, en el alcance inicial se almacenan y se preparan para revisión o retroalimentación auxiliar, evitando que un LLM determine por sí solo la nota oficial.

6.5.3 Falsos negativos en la comparación de salidas

Capítulo 7

Conclusiones

7.1 Análisis del cumplimiento de los objetivos

7.1.1 Grado de cumplimiento del objetivo 1

7.1.2 Grado de cumplimiento del objetivo 2

7.1.3 Grado de cumplimiento del objetivo 3

7.1.4 Grado de cumplimiento del objetivo 4

7.1.5 Grado de cumplimiento del objetivo 5

7.2 Trabajo no realizado y limitaciones del alcance

7.2.1 Integración con LMS (Moodle/Canvas)

7.2.2 Estudio de usabilidad formal (SUS)

7.2.3 Soporte para múltiples lenguajes

7.3 Líneas de trabajo futuro

7.3.1 Mejora de la retroalimentación pedagógica mediante IA no evaluativa

Aunque el prototipo contempla la generación auxiliar de informes anonimizados mediante LLM, queda como línea futura mejorar la calidad pedagógica de dichos informes, evaluar su utilidad real con estudiantes y docentes, y estudiar mecanismos adicionales de validación humana. En ningún caso esta línea debe confundirse con delegar la calificación oficial en un modelo generativo.

7.3.2 Ampliación avanzada del soporte multiformato de preguntas

El soporte inicial de varios tipos de preguntas puede ampliarse con rúbricas más configurables, bancos de preguntas reutilizables, aleatorización controlada, trazas semiautomáticas, análisis de errores frecuentes y compatibilidad con nuevos lenguajes. Esta línea

futura no invalida el alcance inicial multiformato, sino que lo transforma en una base ampliable.

7.3.3 Despliegue en la nube y escalado horizontal

7.3.4 Analítica del proceso de escritura de código

7.4 Valoración personal y cierre

Apéndice A

A.1 Declaración responsable sobre autoría y uso ético de Inteligencia Artificial Generativa

En cumplimiento de la normativa de la [Escuela de Ingeniería Industrial y Aeroespacial de Toledo \(EIIA-To\)](#) ([UCLM](#)) sobre el uso de [Inteligencia Artificial \(IA\)](#) Generativa en [Trabajo Fin de Grado \(TFG\)](#), se declara que en la elaboración de este trabajo se han empleado, con carácter auxiliar, y bajo supervisión y validación personal en todo momento, las siguientes herramientas:

- **Claude Sonnet 5.0** (Anthropic): revisión ortotipográfica y detección de erratas en texto de redacción propia.
- **Codex** (OpenAI): asistencia puntual en la revisión y depuración de fragmentos de código desarrollados por el autor.
- **ChatGPT** (OpenAI): ajustes menores de edición sobre imágenes de elaboración propia.

Estas herramientas no han sustituido la autoría, el criterio técnico ni la validación personal del estudiante. Su uso se ha limitado a tareas auxiliares de revisión puntual, sin delegar en ellas la toma de decisiones técnicas, la interpretación de resultados ni la responsabilidad final sobre el contenido presentado.

Debe distinguirse entre el uso de herramientas de [IA](#) generativa como apoyo externo durante la elaboración de la memoria y la incorporación de un [LLM](#) como funcionalidad experimental del prototipo. En este segundo caso, el modelo se contempla únicamente para generar informes auxiliares de retroalimentación a partir de datos anonimizados o seudonimizados.

$\LaTeX$ es un formato textual para construir documentos complejos, cualquier tipo de documento complejo. No es difícil, y de hecho está pensado para que personas sin formación en tipografía y composición puedan redactar documentos estéticamente impecables. Pero necesita algo de tiempo para acostumbrarse a la filosofía.

Puedes acabar tu TFG simplemente imitando la redacción de este documento, pero seguro que puedes sacar más partido si aprendes algo de $\LaTeX$. Hay muchas guías sencillas

para empezar. Por ejemplo, [ernestoaranda2013] o [gastonsimone2002] son resúmenes suficientemente completos y sencillos. En caso de que quieras ampliar tus conocimientos tendrás que empezar por el libro de Leslie Lamport [leslielamport1994], el creador de $\text{\LaTeX}$. Procura complementarlo con [latexcompanion2004] y, especialmente, navegar por el CTAN (*Comprehensive $\text{\LaTeX}$ Archive Network*). CTAN es un repositorio enorme de paquetes, que facilitan la edición de todo tipo de detalles. Desde las cabeceras de página, hasta la creación de enlaces hipertexto o el manejo de colores.

En este anexo se incluye un pequeño recetario, extraído de la plantilla de Fernando Castillo, que puedes usar a modo de recordatorio rápido, pero procura utilizar una referencia más completa cuando empieces a editar el documento.



Este recetario no es un libro de $\text{\LaTeX}$, ni es para sentarse a leerlo, es para usarlo cuando lo necesites. No te mostramos cómo se escribe en $\text{\LaTeX}$ cada ejemplo, sino que directamente lo hacemos. Cuando necesites hacer algo semejante, mira el código de este documento en https://github.com/UCLM-eiia-to/eiia_doc_tfg_latex

A.2 Texto

En esta sección se muestran ejemplos de efectos de texto.

textbf

Texto en negrita.

textit

Texto en cursiva.

underline

Texto subrayado.

large

Texto grande

Large

Texto más grande

LARGE

Texto mucho más grande

textsc

TEXTO EN VERSALITAS

textsf

Texto en fuente sans-serif

texttt

Texto en fuente mono-espaciada

A veces resulta útil este tipo de texto para funciones de programación o código fuente. Por ejemplo:

```
[x,y]=function(t,z)
```


A.2.1 Listas numeradas y con viñetas

Listas numeradas. Se utilizan cuando la posición que ocupan los elementos es importante, para contar los elementos, o cuando se necesita añadir referencias a algunos de los elementos.

1. diodos
2. transistores
 - (a) transistores pnp
 - (b) transistores npn
 - (c) operacionales
3. operacionales

Listas con viñetas. Se utilizan cuando la posición concreta de los elementos no es importante:

- diodos
- transistores
 - transistores pnp
 - transistores npn
 - operacionales
- operacionales

A.2.2 Acrónimos

Si quieres lista de acrónimos debes marcar los acrónimos con una etiqueta especial. Por ejemplo, así se pondría la primera vez un (). Una vez que se ha usado el acrónimo, ya se puede usar en forma abreviada, como en . Marca incluso en este caso los acrónimos, porque así el PDF permitirá navegar a la definición pinchando sobre él.

A.3 Figuras

En esta sección se muestran algunos ejemplos de figuras. La fig. A.1 es un ejemplo de *Encapsulated PostScript*. Se trata de un formato vectorial, que no se degrada al escalarlo. Este tipo de formatos son los ideales para usar con $\LaTeX$. Utiliza siempre que puedas imágenes en formato EPS o PDF.

La forma de incluir la figura es simple:

```
\begin{figure}
\centering\includegraphics[width=6cm]{ejemplo.eps}
\caption{Figurita ejemplo}
\label{fig:figurita-ejemplo}
\end{figure}
```

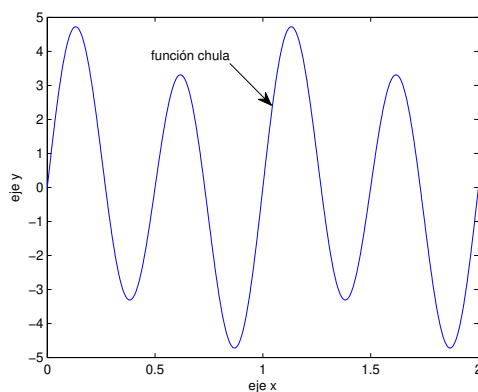


Figura A.1: Figurita ejemplo

El entorno `figure` crea un cuadro flotante, con todo el contenido de la figura, que $\text{\LaTeX}$ coloca en el sitio menos malo. La `caption` es el pie de la figura y la `label` es la etiqueta que nos permitirá referirnos a ella en el texto (con la orden `ref`).

$\text{\LaTeX}$ utiliza un algoritmo nada evidente para colocar las figuras de manera que sea estéticamente agradable. Pero tú puedes influir en las preferencias de colocación. El entorno `figure` tiene un parámetro opcional entre corchetes que indica las opciones de colocación. Por defecto es `[tbp]`, que equivale a *top*, *bottom*, *page*. Eso quiere decir que intenta primero ponerla a comienzo de página. Si no lo consigue, al final de una página. Y si así tampoco lo consigue, en una página entera, solo para la figura. En este ejemplo utilizo las opciones `[hbt]` para que intente la secuencia *here*, *bottom*, *top*, *page*. En este caso prefiero que la ponga debajo antes que arriba, para que no aparezca en una sección anterior.

Con $\text{\LaTeX}$ puedes conseguir que las figuras no se muevan en absoluto, pero eso deja documentos extremadamente descompensados. No lo hagas nunca. Es mejor mover ligeramente la figura en el texto o incluso re-escribir parte del texto, antes de forzar la posición. De todas formas, si no me quieres hacer caso, en la fig. A.2 tienes un ejemplo que fuerza la posición.

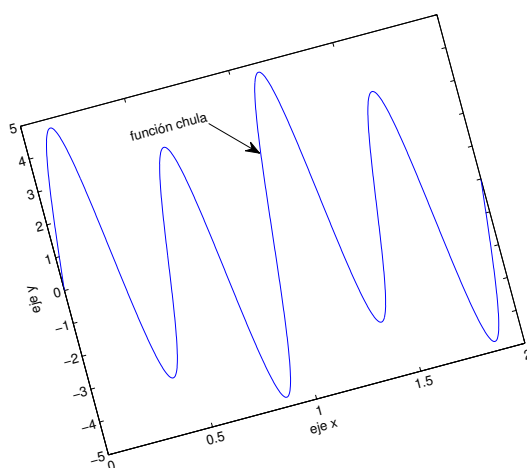


Figura A.2: Figurita ejemplo 2

Para poder poner figuras que no son de elaboración propia es necesario primero obtener permiso del autor y, además, añadir la fuente al pie de foto. Hay muchas guías de estilo

que explican en detalle cómo hacerlo. Por ejemplo, la *American Psychological Association* tiene un [capítulo específico de su manual de publicaciones](#). El manual de la APA se usa extensivamente en todo tipo de literatura científica.

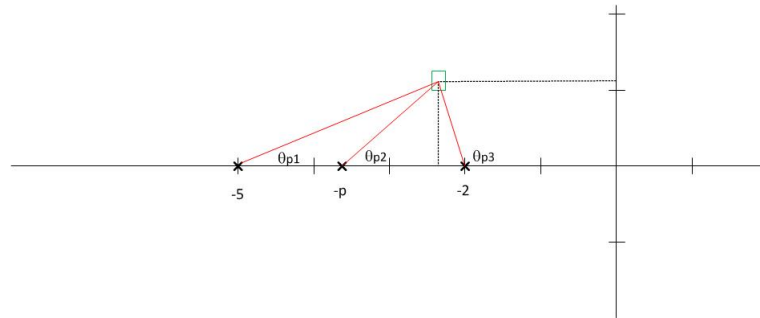


Figura A.3: Figurita ejemplo 3. Extraída de la plantilla de TFG de Fernando Castillo. ©2018 Fernando Castillo. Reproducida con permiso.



Fíjate bien. No se citan las imágenes como si se tratara de referencias bibliográficas. No debe haber pie de página (orden footnote) ni cita (orden cite) en un pie de foto (caption). La atribución de la obra debe estar al mismo nivel que la obra usada. Por eso debe atribuirse completamente en el pie de foto. Si no te gusta como queda haz tus propias imágenes.

Se puede controlar el escalado de la imagen y el ángulo de forma muy sencilla, con las opciones de la orden `includegraphics`. En la fig. A.3 se muestra un ejemplo de figura escalado a un 30 %. La orden `includegraphics` ajusta los parámetros de la imagen para mantener la relación de aspecto original, si esto es posible. Esto hace que podamos especificar simplemente el ancho o el alto deseado, que va a ser lo más habitual. En mi opinión, las opciones más frecuentes de `includegraphics` son, por orden:

width

Fija el ancho de la imagen. Puede ser un tamaño absoluto en centímetros (cm), milímetros (mm) o puntos PostScript (pt). Por ejemplo, `width=1.5cm`. También puede ser un tamaño relativo a cualquier medida del documento. Por ejemplo, `width=0.5\textwidth` sería una figura que ocupe la mitad del ancho del texto.

height

Fija la altura de la imagen. Es similar a `width` pero con la altura. Se puede especificar tanto altura como anchura, de manera que se modifica la relación de aspecto original.

scale

Utiliza un factor de escala para la imagen. Puede ser mayor de 1 para ampliar la imagen.

angle

Gira la imagen un número de grados determinado. Si el número es negativo el giro es en sentido horario. Si es positivo el giro es antihorario.

En la fig. A.5 se muestra un ejemplo de figura girada 30.

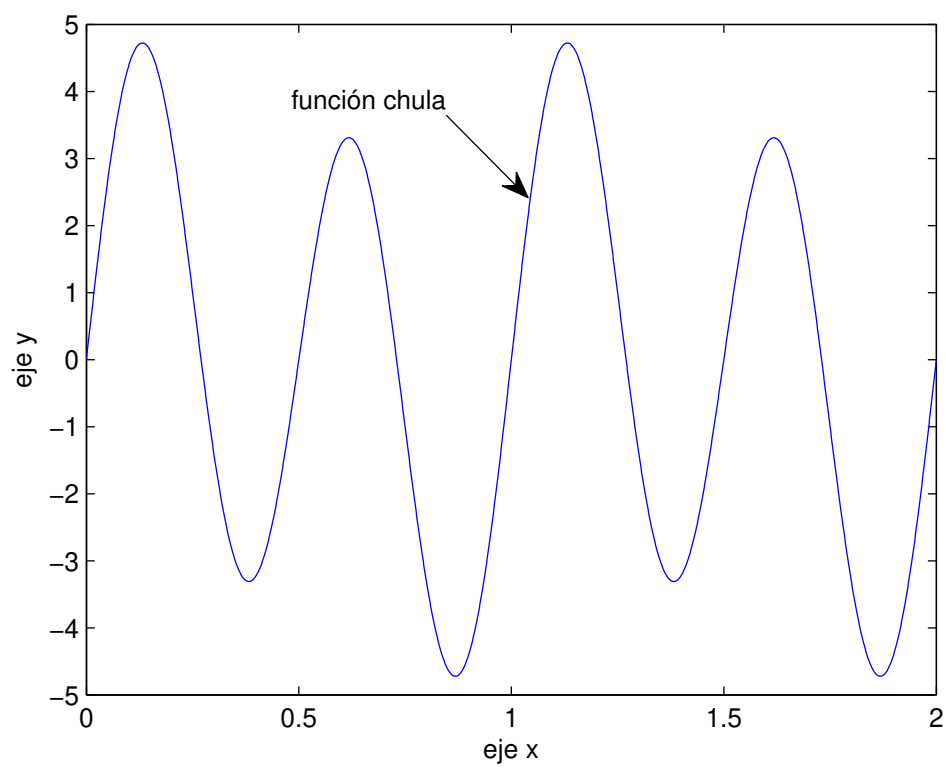


Figura A.4: Figura ejemplo que ocupa todo el ancho del texto.

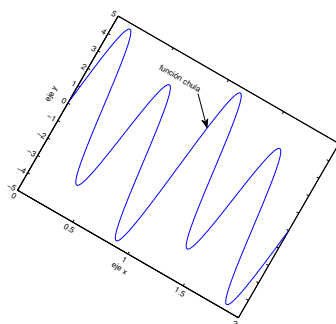


Figura A.5: Figura ejemplo de rotación.



Hay muchos paquetes para definir subfiguras (subfigure, subfig, subfloat, floatrow, subcaption, ...). **Todos son incompatibles entre sí!** En esta plantilla hemos usado el paquete más moderno, subcaption. No mezcles con otros paquetes de subfiguras.

Una característica interesante del entorno figure es que permite definir sub-figuras con la orden subcaptionbox. La colocación de las sub-figuras es prácticamente automática. Un ejemplo puede verse en la figura A.6. Las subfiguras pueden referenciarse etiquetando la *subcaption* (primer argumento de subcaptionbox), como cualquier otro elemento del texto. Por ejemplo, la figura A.6a es igual que la figura A.6b.

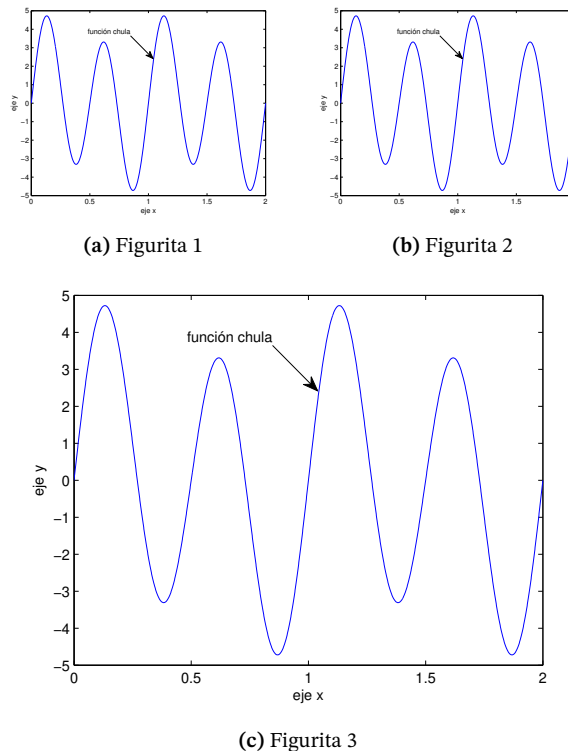


Figura A.6: Matriz de figuras

A.4 Tablas

En esta sección se muestran algunos ejemplos de tablas.

Regulador	Función de Transferencia	orden
P	α_1	2

Tabla A.1: Resultados de la simulación

Las tablas en $\text{\LaTeX}$ no son complejas pero puedes simplificar aún más usando un editor interactivo de tablas. Por ejemplo, en <https://truben.no/table/> hay una aplicación *online* para editar multitud de formatos de tablas. Es especialmente útil para tablas complicadas.

En $\text{\LaTeX}$ es bastante frecuente separar la cabecera del cuerpo de la tabla poniendo dos `hLine`, como en la tabla A.2.

País	Ciudad
España	Madrid
España	Valencia
Francia	París

Tabla A.2: Tabla muy sencilla.

La complejidad empieza cuando hay que expandir celdas para ocupar varias columnas o varias filas. Por ejemplo, la tabla (A.3) tiene una celda multi-columna y otra celda multi-fila. En estos casos un editor interactivo como el de [Peder Lång Skeidsvoll](#) puede ser de gran ayuda para un principiante. Explora las opciones, no son evidentes al principio.

Europa	
País	Ciudad
España	Madrid
	Valencia
Francia	París

Tabla A.3: Fusionando celdas.

Las tablas, al igual que las figuras, tienen un parámetro opcional entre corchetes que indican las preferencias de posición. Se puede forzar pero, al igual que con las figuras, conduce a documentos muy descompensados. Procura evitarlo. Dentro de la tabla se define un entorno `tabular` que indica con su argumento obligatorio las columnas. Este entorno es muy útil en cualquier organización matricial. Se puede usar también para presentar las subfiguras de una figura, o para definir una matriz.

Las tablas muy largas deben dividirse en varias páginas. En el estilo de este TFG hemos incluido el paquete `longtable`, que facilita enormemente escribir este tipo de tablas largas. En ese caso, en lugar del entorno `table` y el entorno `tabular` se usaría solamente el entorno `longtable`, que es una especie de híbrido de los dos, con un montón de características opcionales. Para ilustrar su uso reproducimos [un ejemplo de TeXblog](#) en la tabla A.4.

Tabla A.4: Un ejemplo de tabla larga

Primera	Segunda	Tercera	Cuarta
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4

Continúa en la página siguiente

Tabla A.4 – Continúa de la página anterior

Primera	Segunda	Tercera	Cuarta
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4

A.5 Ecuaciones

Si hay algo donde $\text{\LaTeX}$ es especialmente útil, es en las fórmulas matemáticas. Prácticamente no hay otra opción cuando las fórmulas son relativamente complejas. En esta sección se muestran algunos ejemplos de ecuaciones.

$$\mathbf{v} = \begin{bmatrix} 2 \\ 3 \\ -4 \end{bmatrix} \quad (\text{A.1})$$

En $\text{\LaTeX}$ es trivial el uso de cualquier notación de vectores. Tan solo hay que familiarizarse con las órdenes correspondientes. Por ejemplo, en esta ecuación:

$$\vec{F} = m\vec{a} \quad (\text{A.2})$$

donde $\vec{F}$ es la fuerza, $\vec{a}$ es la actitud y m la masa.

Las ecuaciones pueden referenciarse igual que las figuras, las tablas o las secciones. Por ejemplo, la ecuación A.3 ...

$$\alpha_{\text{inicial}} = \beta^{\text{final}} + \gamma \quad (\text{A.3})$$

Repara bien en cómo se escribe la ecuación anterior. Una palabra (o más de una) en una fórmula debe ponerse con ayuda de las órdenes auxiliares `\mathrm`, `\mathit`, etc. En caso contrario parecerá un conjunto de letras que representan símbolos que se multiplican entre sí. Para más detalles consulta [andrewroberts2004].

$$G(s) = \frac{(s^2 + s + 1)^2}{s^3 + 1} \quad (\text{A.4})$$

El uso de letras griegas o símbolos matemáticos es también muy sencillo. Tan solo hay que familiarizarse con la orden que los inserta. Puede parecer difícil, pero basta con ojear una chuleta como [ésta](#)¹.

La ecuación A.5 muestra un ejemplo de integral. También es muy sencillo, puesto que la notación de los límites coincide con la de los subíndices y superíndices.

$$F(y) = \int_{x_a}^{x_b} K(x, y) f(x) dx \quad (\text{A.5})$$

Cuando se necesita un entorno tabular dentro de un entorno matemático se utiliza el entorno `array`. La ecuación A.6 muestra un ejemplo.

$$\begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix} \quad (\text{A.6})$$

A.6 Bibliografía, citas y referencias

SE DEBERÁ REVISAR ">tex, >latex, >citas-referencias.tex" Otro de los aspectos especialmente cuidados de $\text{\LaTeX}$ es el manejo de bibliografía y citas. En esta plantilla utilizamos el paquete *biblatex*. Es un paquete muy flexible que permite adaptarse a casi cualquier estilo de citas existente. En los documentos de ingeniería no hay consenso en el estilo a utilizar. Nosotros hemos configurado en la plantilla el estilo del IEEE, pero dependiendo del área específica de trabajo puede ser necesario cambiarlo. Consulta con tu tutor.

La bibliografía en $\text{\LaTeX}$ se hace con ayuda de unos archivos auxiliares escritos en formato BibTeX. Es otro formato textual, con una serie de campos que hay que rellenar. Para la composición de entradas BibTeX lo más sencillo es utilizar un editor online, como la página <http://truben.no/latex/bibtex/>. En principio todas las entradas de bibliografía que utilices en tu TFG deben ponerse en `bib/main.bib`.

En $\text{\LaTeX}$ la forma más básica de cita consiste en emplear la orden `cite` con el campo clave que contiene todo registro de BibTeX. Por ejemplo, según el trabajo `[armas2011estimation]` ...mientras que según `[castillo2010design]` el control es una cosa muy buena. Pero hay muchas otras opciones de cita. Consulta la [chuleta de Bib \$\text{\LaTeX}\$](#) para ver todas las posibilidades. Entre las alternativas más frecuentes está la orden `parencite` con el campo clave que contiene el registro BibTeX y entre corchetes la página concreta. Por ejemplo, según `[armas2011estimation]` bla bla. Como ves, el aspecto visual es similar, pero añade la página a la que hacemos referencia. Otra posibilidad es usar la orden `textcite` que añade los autores. Por ejemplo, según `armas2011estimation` bla bla.

Se puede personalizar el aspecto de las citas cambiando los parámetros `style` y `citestyle` del paquete *biblatex* en `sty/eiitfg.cls`. En ingeniería el estilo más utilizado, con mucha diferencia, es el de IEEE, del que existen dos variantes, `ieee` y `ieee-alphabetic`. La única diferencia entre ambos es que en el primero se usan números para identificar las referencias y en el segundo se utilizan iniciales de los apellidos de los autores.

Otros valores muy utilizados son `alphabetic`, `authoryear`, `apa`, `chem-acs`, `m\la`, `phys`, `nature`, `science`. Sin embargo te recomendamos que utilices una de las dos variantes del estilo IEEE porque incluye soporte de entradas BibTeX para patentes.

¹<https://tug.ctan.org/info/symbols/comprehensive/symbols-a4.pdf>

También se puede acceder a campos concretos del registro BibTeX, tales como el autor, el año o el título. Por ejemplo, `armas2011estimation` escribió en `armas2011estimation` el artículo `armas2011estimation`.

Otro estilo más antiguo es el estilo de Vancouver, en el que se usan notas al pie. En $\text{\LaTeX}$ puede hacerse con la orden `footcite`. Por ejemplo, según el profesor Armas² bla bla. Todavía se ve este estilo en libros de historia, pero tiende al desuso porque rompe la linealidad del texto. En general, lo más importante es ser consistente, usa un estilo y solo un estilo en todo el documento.

A.6.1 Lo que toda referencia debe tener

Una referencia bibliográfica se utiliza como argumento de autoridad, para dar peso a tu propia argumentación. Por tanto, hay tres elementos clave que siempre deben estar:

- El autor, puesto que palabras anónimas no dan peso a nada. Recuerda que el autor es lo que da peso a tu argumento. No cites artículos divulgativos, ni autores sin un mínimo prestigio en el campo de lo que afirman.
- El título, puesto que el lector debe poder buscar por sí mismo el documento original.
- La fecha, puesto que un mismo autor puede cambiar de opinión a lo largo de su vida. Por ejemplo, John Maynard Keynes es Premio Nobel, pero tiene numerosos escritos contradictorios. Su opinión era bastante cambiante con el tiempo.

Si falta alguno de estos elementos no es una referencia y no se cita. Se puede poner como una nota a pie de página (footnote) o como una URL en el cuerpo del texto, pero no como una referencia.

Por cierto, es conveniente citar las fuentes. Es decir, debes tomarte la molestia de buscar quién dijo o inventó lo que citas y dónde lo publicó por primera vez. Es la mínima cortesía que se debe tener con los colegas de profesión. Supongo que tú también querrás crédito por tu trabajo en tu futuro profesional.

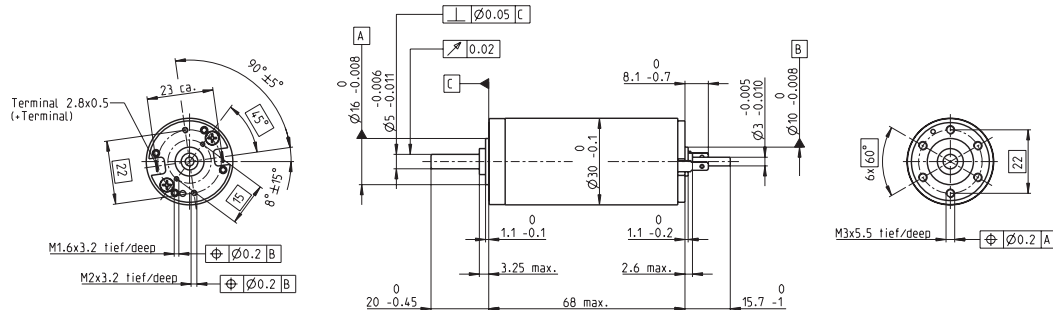
A.7 Hojas de datos

Las hojas de datos normalmente son archivos PDF. En $\text{\LaTeX}$ hay dos formas de insertar archivos PDF. La más sencilla es utilizar la orden `includegraphics` que ya hemos visto en las figuras, pero solo se puede incrustar una página en cada llamada a `includegraphics`.

Si el documento tiene varias páginas habría que insertar cada una de las páginas de interés con órdenes `includegraphics` independientes. En el parámetro `page` podemos elegir la página a insertar. Este método nos da el máximo control para poder poner pies de figura y el posicionamiento. Sin embargo, cuando el documento tiene muchas páginas es un poco engorroso. Para esos casos se puede utilizar la orden `includepdf` del paquete `pdfpages`.

Para ilustrarlo veamos el mismo ejemplo, primero con `includepdf` y luego con `includegraphics`.

²`armas2011estimation`.

RE 30 Ø30 mm, Graphite Brushes, 60 Watt**M 1:2**

- Stock program
 Standard program
 Special program (on request)

according to dimensional drawing
shaft length 15.7 shortened to 8.7 mm

Part Numbers

310005	310006	310007	310008	310009
268193	268213	268214	268215	268216

Motor Data**Values at nominal voltage**

1 Nominal voltage	V	12	18	24	36	48
2 No load speed	rpm	8170	8590	8810	8590	8490
3 No load current	mA	301	213	165	106	78.6
4 Nominal speed	rpm	7630	7910	8050	7840	7760
5 Nominal torque (max. continuous torque)	mNm	51.6	75.5	85.6	86.6	89.7
6 Nominal current (max. continuous current)	A	4	4	3.47	2.28	1.74
7 Stall torque	mNm	852	1000	1020	1000	1050
8 Starting current	A	61.1	50.3	39.3	25.2	19.6
9 Max. efficiency	%	85	87	87	87	88
Characteristics						
10 Terminal resistance	Ω	0.196	0.358	0.611	1.43	2.45
11 Terminal inductance	mH	0.0345	0.0703	0.119	0.281	0.513
12 Torque constant	mNm/A	13.9	19.9	25.9	39.8	53.8
13 Speed constant	rpm/V	685	479	369	240	178
14 Speed / torque gradient	rpm/mNm	9.64	8.61	8.69	8.61	8.09
15 Mechanical time constant	ms	3.4	3.24	3.05	2.98	2.94
16 Rotor inertia	gcm ²	33.7	35.9	33.5	33.1	34.7

Specifications**Thermal data**

17 Thermal resistance housing-ambient	6.0 K/W
18 Thermal resistance winding-housing	1.7 K/W
19 Thermal time constant winding	16.3 s
20 Thermal time constant motor	525 s
21 Ambient temperature	-30...+100°C
22 Max. permissible winding temperature	+125°C

Mechanical data (ball bearings)

23 Max. permissible speed	12000 rpm
24 Axial play	0.05 - 0.15 mm
25 Radial play	0.025 mm
26 Max. axial load (dynamic)	5.6 N
27 Max. force for press fits (static) (static, shaft supported)	110 N
28 Max. radial load, 5 mm from flange	1200 N
	28 N

Other specifications

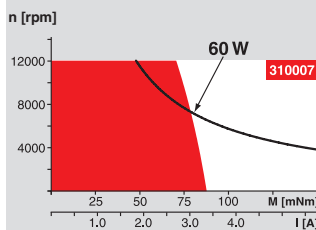
29 Number of pole pairs	1
30 Number of commutator segments	13
31 Weight of motor	260 g

Values listed in the table are nominal.
Explanation of the figures on page 79.

△ Tolerances may vary from the standard specification.

Option

Preloaded ball bearings

Operating Range**Comments**

- Continuous operation**
In observation of above listed thermal resistance (lines 17 and 18) the maximum permissible winding temperature will be reached during continuous operation at 25°C ambient.
= Thermal limit.
- Short term operation**
The motor may be briefly overloaded (recurring).
- **Assigned power rating**

maxon Modular System**Planetary Gearhead**

Ø32 mm

0.75 - 6.0 Nm

Page 272-278

Koaxdrive

Ø32 mm

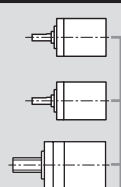
1.0 - 4.5 Nm

Page 281

Spindle Drive

Ø32 mm

Page 301-303

**Overview on page 20-25**

Encoder MR
256 - 1024 CPT,
3 channels
Page 320

Encoder HED_ 5540
500 CPT,
3 channels
Page 325/329

Recommended Electronics:

ESCON 36/2 DC	Page 342
ESCON Module 50/5	343
ESCON 50/5	344
ESCON 70/10	344
EPOS2 Module 36/2	350
EPOS2 24/5, EPOS2 50/5	351
EPOS2 P 24/5	354
EPOS3 70/10 EtherCAT	357
MAXPOS 50/5	360
Notes	22

La opción con includegraphics permite referirse a una página concreta y poner pies de figura. Por ejemplo, en la figura A.7 se muestra la hoja de especificaciones del motor empleado.

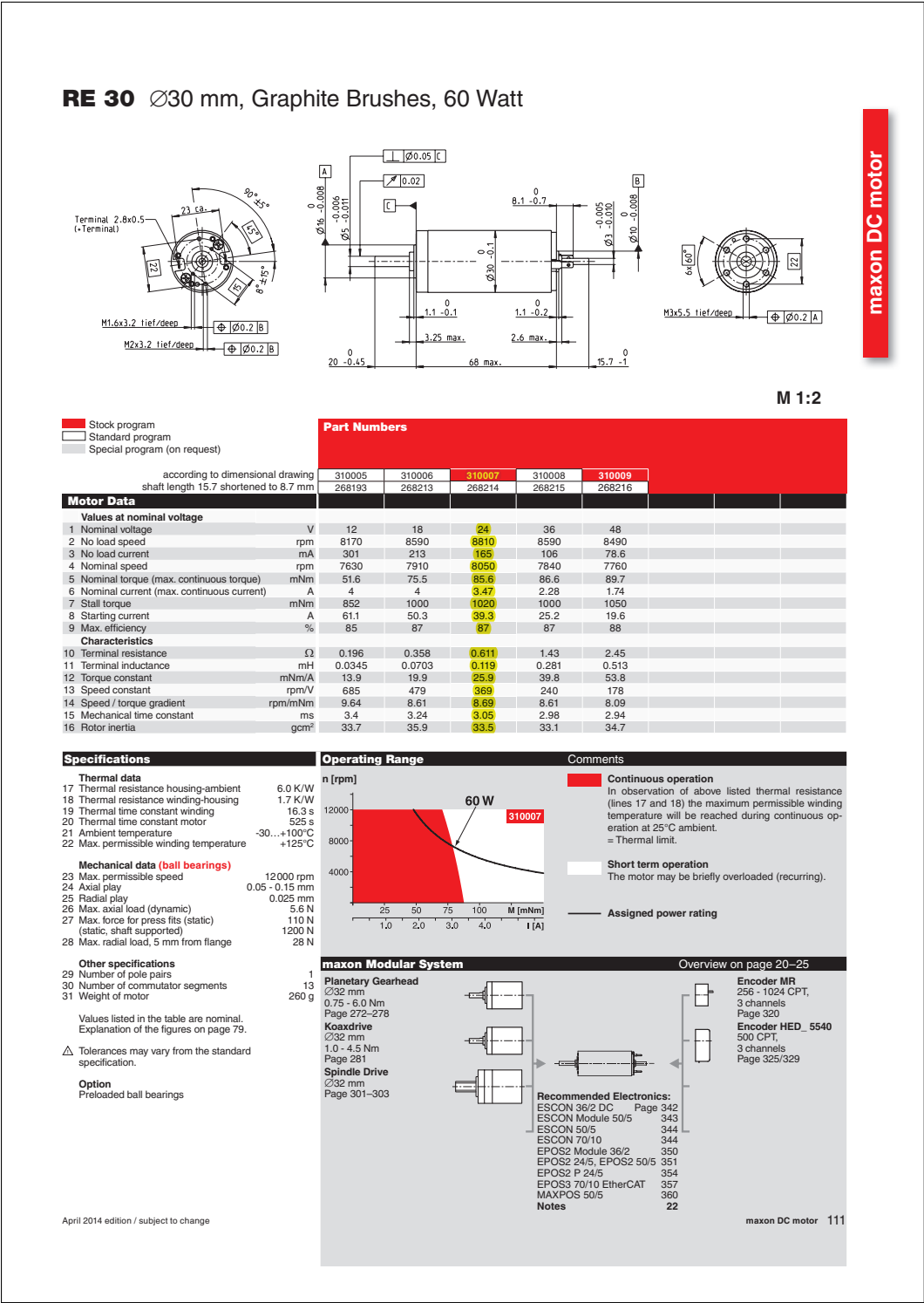


Figura A.7: Figura ejemplo. Tomada de hoja de catálogo de motores DC con escobillas de grafito de Maxon © 2014 Maxon Motors. Reproducida con permiso.

A.8 Código fuente

Para insertar fragmentos o listados completos de código se puede usar el paquete `lstlistings`. Permite incluir archivos o parte de archivos directamente del proyecto con la orden `lstinputlisting`.

Listado A.1: Ejercicio 20 como texto incorporado

```

1 function [T]=Ejercicio20(f,c)
2
3 % aquí hay un comentario con eñes
4
5 T = char('B'*ones(8,8));
6
7 for i=1:8
8     for j=1:8
9         if ( (i==f) || (j==c) || (i+j==f+c) || (i-j==f-c) )
10             T(i,j)='*';
11         elseif ( rem(i+j,2)~=0 )
12             T(i,j)='N';
13         end
14     end
15 end
16
17 T(f,c)='R';

```

O bien se puede copiar el texto del programa o fragmento en un entorno `lstlisting` con las mismas opciones que la orden `lstinputlisting`.

Listado A.2: Ejercicio 20 como texto en línea.

```

1 function [T]=Ejercicio20(f,c)
2
3 T = char('B'*ones(8,8));
4
5 for i=1:8
6     for j=1:8
7         if ( (i==f) || (j==c) || (i+j==f+c) || (i-j==f-c) )
8             T(i,j)='*';
9         elseif ( rem(i+j,2)~=0 )
10             T(i,j)='N';
11         end
12     end
13 end
14
15 T(f,c)='R';

```



Te recomendamos que incluyas los archivos o parte de los archivos directamente del código de tu proyecto, ya sea mediante `lstinputlisting` o mediante `inputminted`. De esta forma mantendrás sincronizado el documento con el código fuente.

El paquete `lstlisting` te permite quitar los números y el marco, cuando el código se incluye como parte del texto.

```

function [T]=Ejercicio20(f,c)

T = char('B'*ones(8,8));

```

```

for i=1:8
    for j=1:8
        if ( (i==f) || (j==c) || (i+j==f+c) || (i-j==f-c) )
            T(i,j)='*';
        elseif ( rem(i+j,2)~=0 )
            T(i,j)='N';
        end
    end
end

T(f,c)='R';

```

Otra forma de incluir código es mediante el entorno `verbatim`. Este método no tiene resalte de sintaxis ni facilidades de ningún tipo para definir etiquetas o numerar las líneas.

```

function [T]=Ejercicio20(f,c)

T = char('B'*ones(8,8));

for i=1:8
    for j=1:8
        if ( (i==f) || (j==c) || (i+j==f+c) || (i-j==f-c) )
            T(i,j)='*';
        elseif ( rem(i+j,2)~=0 )
            T(i,j)='N';
        end
    end
end

T(f,c)='R';

```

Otra forma alternativa a `lstlisting` es el paquete `minted`, que colorea el programa según el lenguaje empleado.

```

1  function [T]=Ejercicio20(f,c)
2
3  T = char('B'*ones(8,8));
4
5  for i=1:8
6      for j=1:8
7          if ( (i==f) || (j==c) || (i+j==f+c) || (i-j==f-c) )
8              T(i,j)='*';
9          elseif ( rem(i+j,2)~=0 )
10             T(i,j)='N';
11          end
12      end
13  end
14
15  T(f,c)='R';

```

O bien, usando la orden `inputminted` para incluir directamente un archivo Matlab.

```

1  function [T]=Ejercicio20(f,c)

```

```

2
3 % aquí hay un comentario con eñes
4
5 T = char('B'*ones(8,8));
6
7 for i=1:8
8     for j=1:8
9         if ( (i==f) || (j==c) || (i+j==f+c) || (i-j==f-c) )
10             T(i,j)='*';
11         elseif ( rem(i+j,2)~=0 )
12             T(i,j)='N';
13         end
14     end
15 end
16
17 T(f,c)='R';

```

A.9 Compilación local

Si utilizas Overleaf con una cuenta gratuita es muy posible que excedas el límite de tiempo para compilar. Nuestra recomendación es que tu tutor cree el proyecto correspondiente a tu TFG y te lo comparta como colaborador con permiso de escritura. Pero también puedes compilarlo localmente en tu propio ordenador instalando algo de software. Como ha sido una preocupación recurrente, hemos cambiado la plantilla para que sea más fácil de compilar en Microsoft Windows.

1. Instala [TexStudio](#). Es un editor de $\text{\LaTeX}$ no tan sencillo como Overleaf pero mucho más amigable que editar los archivos en un editor de texto normal.
2. Instala [MikTeX](#). Si marcas la opción *Install missing packages on the fly* los paquetes de $\text{\LaTeX}$ se instalarán automáticamente, que resulta muy conveniente.
3. Si usas Windows instala [Strawberry Perl](#). Es necesario para utilizar la orden *Latexmk* en la compilación.
4. Instala [Python](#). Lo necesitarás si usas el entorno **minted** para colorear trozos de código. A la hora de instalar Python no olvides marcar la casilla para añadir Python a la variable de entorno Path.

A.9.1 Configurar el entorno

1. Abre la aplicación *MikTeX Console* e instala las fuentes de la plantilla. Para ello pincha en *Packages* y añade los paquetes *stix*, *esstix*, *stix2-type1*, *stix2-otf*, *arial* y *noto*.
2. Añade la carpeta que contiene *perl.exe* a la variable de entorno Path. Si instalaste Strawberry Perl en el lugar por omisión debes añadir *C:\Strawberry\perl\bin*.
3. Elimina el alias de ejecución de Python de Windows. Para ello pulsa la tecla de Windows y escribe *alias*, selecciona *Administrar alias de ejecución de aplicaciones*. Asegúrate que el alias del *Instalador de aplicaciones* no está activado para *python.exe* y *python3.exe*.

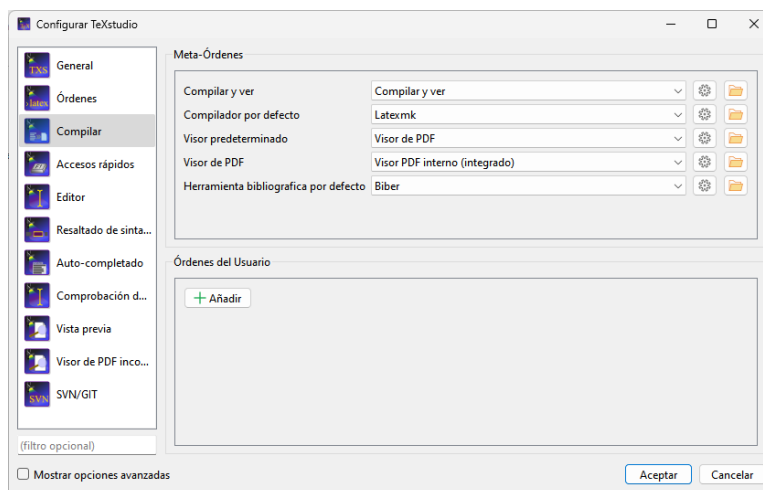
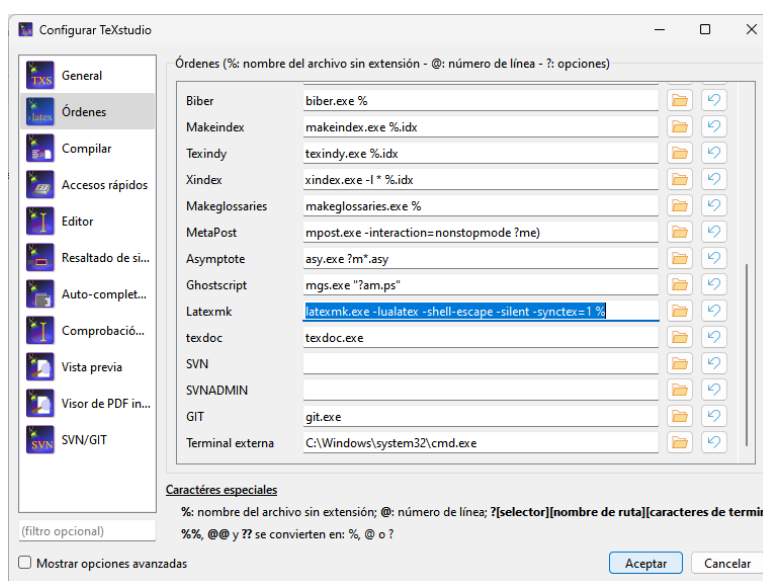


Figura A.8: Configuración de la compilación en TeXStudio

Figura A.9: Configuración de la orden *Latexmk* en TeXStudio

Comprueba que todo está correctamente configurado. Para ello abre una ventana de órdenes (PowerShell o Símbolo de sistema) y teclea `perl -v` y posteriormente `python -V`. Ambas órdenes deberían ejecutarse sin error y mostrar las versiones de Perl y Python respectivamente. Por último ejecuta `pip install Pygments` que instalará el software de coloreado de código.

A.9.2 Configurar TeXStudio

Entra en *Opciones* → *Configurar TeXStudio* y configura la sección *Compilar* como se muestra en la Figura A.8. El compilador por defecto debe ser **Latexmk** y la herramienta bibliográfica debe ser **Biber**.

Entra en la sección *Órdenes* y modifica las opciones de *Latexmk* como muestra la Figura A.9. La opción `-pdf` debe ser sustituida por `-luatex -shell-escape`.

Una vez configurado, para compilar el documento solo hay que pulsar en el botón **Compilar** . Para previsualizar el resultado hay que pulsar sobre el botón **Visualizar** .

Bibliografía

- [ABI+20] A. Agache et al., «Firecracker: Lightweight Virtualization for Serverless Applications», en *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, USENIX Association, 2020, págs. 419-434. visitado 3 de jun. de 2026. dirección: <https://www.usenix.org/conference/nsdi20/presentation/agache>.
- [Ala05] K. M. Ala-Mutka, «A Survey of Automated Assessment Approaches for Programming Assignments», *Computer Science Education*, vol. 15, nº 2, págs. 83-102, 2005. DOI: [10.1080/08993400500150747](https://doi.org/10.1080/08993400500150747).
- [BKM09] A. Bangor, P. Kortum y J. Miller, «Determining What Individual SUS Scores Mean: Adding an Adjective Rating Scale», *Journal of Usability Studies*, vol. 4, nº 3, págs. 114-123, 2009.
- [Bot25] A. Boté Ribalta, «testBench: Automated management and evaluation platform of code projects in education», Bachelor's thesis, Universitat Politècnica de Catalunya (UPC), Escola d'Enginyeria de Telecomunicació i Aeroespacial de Castelldefels, 20 de jun. de 2025. visitado 3 de jun. de 2026. dirección: <https://upcommons.upc.edu/entities/publication/51ce76cb-d6d9-40a6-a407-b83b3a85eeb1>.
- [Bro96] J. Brooke, «SUS: A Quick and Dirty Usability Scale», en *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, I. L. McClelland y B. Weerdmeester, eds., Taylor & Francis, 1996, págs. 189-194.
- [CD13] J. C. Caiza y J. M. Del Alamo, «Programming Assignments Automatic Grading: Review of Tools and Implementations», en *Proceedings of the 7th International Technology, Education and Development Conference (INTED2013)*, Valencia, Spain: IATED, 2013, págs. 5691-5700.
- [Car25] P. Carbonnelle. «PYPL PopularitY of Programming Language Index, January 2025», visitado 15 de ene. de 2026. dirección: <https://pyp1.github.io/PYPL.html>.
- [Čep24] K. Čepelis, «The automation of grading programming exams in computer science education», Bachelor's thesis, University of Twente, Faculty of Electrical Engineering, Mathematics y Computer Science, 5 de jul. de 2024. visitado 3 de jun. de 2026. dirección: https://essay.utwente.nl/fileshare/file/100793/Cepelis_BA_EEMCS.pdf.
- [CKLO03] B. Cheang, A. Kurnia, A. Lim y W.-C. Oon, «On Automated Grading of Programming Assignments in an Academic Institution», *Computers & Education*, vol. 41, nº 2, págs. 121-131, 2003. DOI: [10.1016/S0360-1315\(03\)00030-7](https://doi.org/10.1016/S0360-1315(03)00030-7).
- [Cod26] Codecademy, *Best Python Courses + Tutorials*, <https://www.codecademy.com/catalog/language/python>, Accessed: 2026-06-05, 2026.

- [Cod24] Codeforces. «Codeforces: Competitive Programming Platform», visitado 15 de ene. de 2026. dirección: <https://codeforces.com>.
- [Com22] Computing Research Association. «CRA Taulbee Survey: Computing Degree and Enrollment Trends», visitado 10 de ene. de 2026. dirección: <https://cra.org/data/>.
- [Cou26] Coursera, *Best Python Courses & Certificates*, <https://www.coursera.org/courses?query=python>, Accessed: 2026-06-05, 2026.
- [DKG23] P. Denny, V. Kumar y N. Giacaman, «Conversational Difficulties with ChatGPT When Learning to Program», en *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE '23)*, Toronto, Canada: ACM, 2023, págs. 1239-1245. DOI: [10.1145/3545945.3569823](https://doi.org/10.1145/3545945.3569823).
- [Doc24] Docker Inc. «Docker Security Best Practices», visitado 15 de feb. de 2025. dirección: <https://docs.docker.com/engine/security/>.
- [Edw03] S. H. Edwards, «Teaching Software Testing: Automatic Grading Meets Test-First Coding», en *Companion of the 18th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '03)*, Anaheim, CA, USA: ACM, 2003, págs. 318-319. DOI: [10.1145/949344.949431](https://doi.org/10.1145/949344.949431).
- [Eur23] European Commission. «Europe's Digital Decade: Digital Targets for 2030», visitado 10 de ene. de 2026. dirección: <https://digital-strategy.ec.europa.eu/en/policies/europes-digital-decade>.
- [Eur16] European Parliament and Council of the European Union, *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data (General Data Protection Regulation)*, 27 de abr. de 2016. visitado 3 de jun. de 2026. dirección: <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>.
- [GK99] M. Gordon y M. Kowalski, «Toward Automated Assessment in Computer Science», en *Proceedings of the 1999 Annual Joint Conference of the NEACI-SAM*, New York, USA, 1999.
- [Graa] Gradescope. «Gradescope Autograder Documentation», visitado 2 de jun. de 2026. dirección: <https://gradescope-autograders.readthedocs.io/>.
- [Grab] Gradescope. «Using Gradescope LTI 1.3 with Canvas as an Instructor», visitado 2 de jun. de 2026. dirección: <https://guides.gradescope.com/hc/en-us/articles/23586543164173-Using-Gradescope-LTI-1-3-with-Canvas-as-an-instructor>.
- [gVi] gVisor. «What Is gVisor?», visitado 2 de jun. de 2026. dirección: <https://gvisor.dev/docs/>.
- [Hac24] HackerRank. «HackerRank: Technical Skills Assessment Platform», visitado 15 de ene. de 2026. dirección: <https://www.hackerrank.com>.
- [HLB10] O. R. Harmon, J. Lambrinos y J. Buffolino, «Assessment Design and Cheating Risk in Online Instruction», *Online Journal of Distance Learning Administration*, vol. 13, n° 3, 2010. visitado 3 de jun. de 2026. dirección: https://ojdla.com/archive/fall133/harmon_lambrinos_buffolino133.pdf.
- [Hol60] J. Hollingsworth, «Automatic Graders for Programming Classes», *Communications of the ACM*, vol. 3, n° 10, págs. 528-529, 1960. DOI: [10.1145/367415.367422](https://doi.org/10.1145/367415.367422).

- [IAKS10] P. Ihanola, T. Ahoniemi, V. Karavirta y O. Seppälä, «Review of Recent Systems for Automatic Assessment of Programming Assignments», en *Proceedings of the 10th Koli Calling International Conference on Computing Education Research*, Koli, Finland: ACM, 2010, págs. 86-93. DOI: [10.1145/1930464.1930480](https://doi.org/10.1145/1930464.1930480).
- [JGB05] M. Joy, N. Griffiths y R. Boyatt, «The BOSS Online Submission and Assessment System», *Journal on Educational Resources in Computing*, vol. 5, nº 3, 2005, artno 2. DOI: [10.1145/1163405.1163407](https://doi.org/10.1145/1163405.1163407).
- [Kat24] Kattis. «Kattis Problem Archive», visitado 2 de jun. de 2026. dirección: <https://open.kattis.com/>.
- [KCM+23] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop y T. Grossman, «Studying the Effect of AI Code Generators on Supporting Novice Learners in Introductory Programming», en *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, Hamburg, Germany: ACM, 2023, págs. 1-23. DOI: [10.1145/3544548.3580919](https://doi.org/10.1145/3544548.3580919).
- [KC07] B. A. Kitchenham y S. Charters, *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Keele, UK: Keele University y Durham University, 2007, Technical Report EBSE 2007-001.
- [Lee24] LeetCode. «LeetCode: Online Coding Platform», visitado 15 de ene. de 2026. dirección: <https://leetcode.com>.
- [18] *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales*, 5 de dic. de 2018. visitado 2 de jun. de 2026. dirección: <https://www.boe.es/buscar/act.php?id=B0E-A-2018-16673>.
- [LH16] R. Lobb y J. Harlow, «Coderunner: A Tool for Assessing Computer Programming Skills in the Moodle E-Learning Platform», *ACM Inroads*, vol. 7, nº 1, págs. 47-51, 2016. DOI: [10.1145/2810041](https://doi.org/10.1145/2810041).
- [Mar21] M. Mares. «Sandboxing in Python: Why It's Hard and How to Bypass It», visitado 15 de feb. de 2025. dirección: <https://book.hacktricks.xyz/misc/basic-python/bypass-python-sandboxes>.
- [MDN26a] MDN Web Docs. «Document: Visibilitychange Event», visitado 3 de jun. de 2026. dirección: https://developer.mozilla.org/en-US/docs/Web/API/Document/visibilitychange_event.
- [MDN26b] MDN Web Docs. «Window: Blur Event», visitado 3 de jun. de 2026. dirección: https://developer.mozilla.org/en-US/docs/Web/API/Window/blur_event.
- [MBKS25] M. Messer, N. C. C. Brown, M. Kölling y M. Shi, «How Consistent Are Humans When Grading Programming Assignments?», *ACM Transactions on Computing Education*, vol. 25, nº 4, págs. 1-37, 2025, artno 49. DOI: [10.1145/3759256](https://doi.org/10.1145/3759256).
- [Nie00] J. Nielsen. «Why You Only Need to Test with 5 Users», visitado 2 de jun. de 2026. dirección: <https://www.nngroup.com/articles/why-you-only-needed-to-test-with-5-users/>.
- [nsj] nsjail. «NsJail: Linux Process Isolation Tool», visitado 2 de jun. de 2026. dirección: <https://nsjail.dev/>.
- [Ope24] Open edX. «CodeJail: Sandboxing Python Execution», visitado 15 de feb. de 2025. dirección: <https://github.com/openedx/codejail>.

- [PLF22] J. C. Paiva, J. P. Leal y Á. Figueira, «Automated Assessment in Computer Science Education: A State-of-the-Art Review», *ACM Transactions on Computing Education*, vol. 22, nº 3, págs. 1-40, 2022. DOI: [10.1145/3513140](https://doi.org/10.1145/3513140).
- [PRH15] Á. Paredes-García, J. C. Rodríguez-del-Pino y Z. J. Hernández-Figueroa, «Integración de herramientas de evaluación automática en el proceso de enseñanza-aprendizaje de la programación», *Revista Iberoamericana de Educación en Tecnología y Tecnología en Educación*, vol. 15, págs. 16-23, 2015.
- [PH06] D. Parsons y P. Haden, «Parson's Programming Puzzles: A Fun and Effective Learning Tool for First Programming Courses», en *Proceedings of the 8th Australasian Conference on Computing Education*, ép. Conferences in Research and Practice in Information Technology, vol. 52, Australian Computer Society, Inc., 2006, págs. 157-163.
- [Peñ15] R. Peña, «Python como primera aproximación a la programación», *ReVisión*, vol. 8, nº 2, págs. 143-150, 2015. visitado 2 de jun. de 2026. dirección: https://aenui.org/revision/pdf.php?f=2015_8_2_143.pdf.
- [PVM+23] T. Phung, J. Vietri, T. Mitchell, W. Weimer, M. Hilton y C. Timperley, «Generating Automated Feedback on Student Code with GPT-4», en *Proceedings of the 2023 ACM International Computing Education Research Conference*, Chicago, IL, USA: ACM, 2023, págs. 307-321. DOI: [10.1145/3568813.3600130](https://doi.org/10.1145/3568813.3600130).
- [Pie13] V. Pieterse, «Automated Assessment of Programming Assignments», en *Proceedings of the 3rd Computer Science Education Research Conference on Computer Science Education Research (CSERC '13)*, Arnhem, The Netherlands: Open Universiteit, 2013, págs. 45-56. visitado 3 de jun. de 2026. dirección: <https://dl.acm.org/doi/10.5555/2541917.2541921>.
- [PyC24] PyCQA. «Pylint: Static Code Analysis for Python», visitado 15 de ene. de 2026. dirección: <https://pylint.pycqa.org>.
- [QL12] R. Queirós y J. P. Leal, «Programming Exercises Evaluation Systems: An Interoperability Survey», en *Proceedings of the 4th International Conference on Computer Supported Education (CSEDU 2012)*, SciTePress, 2012, págs. 83-90, ISBN: 978-989-8565-06-8. DOI: [10.5220/0003924900830090](https://doi.org/10.5220/0003924900830090).
- [22] *Real Decreto 217/2022, de 29 de marzo, por el que se establece la ordenación y las enseñanzas mínimas de la Educación Secundaria Obligatoria*, 29 de mar. de 2022. visitado 2 de jun. de 2026. dirección: <https://www.boe.es/buscar/act.php?id=B0E-A-2022-4975>.
- [RRH12] J. C. Rodríguez-del-Pino, E. Rubio-Royo y Z. J. Hernández-Figueroa, «A Virtual Programming Lab for Moodle with Automatic Assessment and Anti-Plagiarism Features», en *Proceedings of the 2012 International Conference on e-Learning, e-Business, Enterprise Information Systems, and e-Government (EEE '12)*, Las Vegas, NV, USA, 2012, págs. 117-123.
- [SKGA17] A. Singh, S. Karayev, K. Gutowski y P. Abbeel, «Gradescope: A Fast, Flexible, and Fair System for Scalable Assessment of Handwritten Work», en *Proceedings of the 4th ACM Conference on Learning at Scale (L@S '17)*, Cambridge, MA, USA: ACM, 2017, págs. 81-88. DOI: [10.1145/3051457.3051466](https://doi.org/10.1145/3051457.3051466).
- [SGS13] R. Singh, S. Gulwani y A. Solar-Lezama, «Automated Feedback Generation for Introductory Programming Assignments», en *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*, Seattle, WA, USA: ACM, 2013, págs. 15-26. DOI: [10.1145/2491956.2462195](https://doi.org/10.1145/2491956.2462195).

- [SHP+06] J. Spacco, D. Hovemeyer, W. Pugh, F. Emad, J. K. Hollingsworth y N. Padua-Perez, «Experiences with Marmoset: Designing and Using an Advanced Submission and Testing System for Programming Courses», en *Proceedings of the 11th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE '06)*, Bologna, Italy: ACM, 2006, págs. 13-17. DOI: [10.1145/1140124.1140131](https://doi.org/10.1145/1140124.1140131).
- [Sta23] Stack Overflow. «Stack Overflow Developer Survey 2023», visitado 15 de ene. de 2026. dirección: <https://survey.stackoverflow.co/2023/>.
- [Sub14] Submittity project (Rensselaer Polytechnic Institute). «Submittity: Homework Submission, Automated Grading and TA Grading System», visitado 10 de feb. de 2026. dirección: <https://github.com/Submittity/Submittity>.
- [TIO25] TIOBE Software. «TIOBE Index for January 2025», visitado 15 de ene. de 2026. dirección: <https://www.tiobe.com/tiobe-index/>.
- [Tur18] Turnitin. «Turnitin Acquires Gradescope», visitado 2 de jun. de 2026. dirección: <https://www.turnitin.com/press/turnitin-acquires-gradescope>.
- [WFH21] N. Weinman, A. Fox y M. A. Hearst, «Improving Instruction of Programming Patterns with Faded Parsons Problems», en *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021. DOI: [10.1145/3411764.3445292](https://doi.org/10.1145/3411764.3445292).

